КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп’ютерних наук та кібернетики

Звіт

до Лабораторної роботи 3

з дисципліни

«Методи обробки мультимедійної інформації»

Виконав студент 2-го курсу
ОНП «Штучний Інтелект»
Ходаков Максим Олегович

Київ – 2026

РЕФЕРАТ

Звіт про лабораторну роботу: 55 с., 10 рис., 8 табл., 19 джерел, 2 додатки.

Об'єкт дослідження — процес обробки відео для виявлення та розпізнавання об'єктів дорожньої сцени.

Предмет дослідження — методи виявлення та відстеження об'єктів, розпізнавання номерних знаків та ідентифікації осіб на основі попередньо навчених нейронних мереж.

Мета — розробити елементи програмного забезпечення, що виділяють і розпізнають об'єкти у відеофільмі, визначають номер автомобіля, ідентифікують водія та визначають рухомий об'єкт.

Методи: згорткові нейронні мережі YOLO11m та YOLOv11-LP, трекер ByteTrack, оптичний потік Лукаса–Канаде з RANSAC, OCR EasyOCR з постобробкою за шаблоном номера та голосуванням по кадрах, детектор облич YuNet і мережа розпізнавання облич SFace.

Результати: на синтетичному відео тривалістю 40 с з еталонною розміткою виявлено й розпізнано 5 об'єктів сцени, номер автомобіля KA7429MB прочитано точно в 100 % кадрів з відкритим знаком, водія ідентифіковано із середньою косинусною схожістю 0,707, єдиний рухомий об'єкт (пасажир) визначено завдяки компенсації руху камери.

ОБРОБКА ВІДЕО, ВИЯВЛЕННЯ ОБ'ЄКТІВ, YOLO11, BYTETRACK, РОЗПІЗНАВАННЯ НОМЕРНИХ ЗНАКІВ, EASYOCR, РОЗПІЗНАВАННЯ ОБЛИЧ, SFACE, ОПТИЧНИЙ ПОТІК, КОМПЕНСАЦІЯ РУХУ КАМЕРИ

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СИМВОЛІВ, ОДИНИЦЬ, СКОРОЧЕНЬ І ТЕРМІНІВ.....5
ВСТУП.....7
1 ПОСТАНОВКА ЗАДАЧІ ТА ВИХІДНІ ДАНІ.....9
1.1 Умова завдання.....9
1.2 Опис відео.....9
1.3 Еталонна розмітка.....11
2 ТЕОРЕТИЧНІ ВІДОМОСТІ ТА ВИБІР МОДЕЛЕЙ.....13
2.1 Виявлення об'єктів. YOLO11.....13
2.2 Відстеження об'єктів. ByteTrack.....13
2.3 Оцінка руху камери.....14
2.4 Розпізнавання номерних знаків.....14
2.5 Виявлення та розпізнавання облич. YuNet і SFace.....15
2.6 Обґрунтування вибору моделей.....15
3 ПРОГРАМНА РЕАЛІЗАЦІЯ.....17
3.1 Середовище та структура проєкту.....17
3.2 Схема конвеєра обробки.....17
3.3 Виявлення та відстеження об'єктів.....18
3.4 Компенсація руху камери.....19
3.5 Розпізнавання номерного знака.....20
3.6 Ідентифікація водія.....21
3.7 Прийняття рішень по треках.....22
3.8 Анотоване відео та оцінювання.....24
4 РЕЗУЛЬТАТИ.....25
4.1 Виявлені об'єкти.....25
4.2 Рухомий об'єкт.....26
4.3 Номер автомобіля.....29
4.4 Ідентифікація водія.....30
4.5 Кількісні показники.....31

5 АНАЛІЗ ПОМИЛОК ТА СПОСОБИ ПОКРАЩЕННЯ.....33
ВИСНОВКИ.....35
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ.....37
ДОДАТОК А Вихідний код основних модулів.....39
ДОДАТОК Б Тези доповіді.....55

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СИМВОЛІВ, ОДИНИЦЬ, СКОРОЧЕНЬ І ТЕРМІНІВ

CLAHE	– Contrast Limited Adaptive Histogram Equalization, адаптивне вирівнювання гістограми з обмеженням контрасту
COCO	– Common Objects in Context, набір даних для виявлення об'єктів (80 класів)
CRAFT	– Character Region Awareness for Text detection, детектор тексту
CRNN	– Convolutional Recurrent Neural Network, згорткова рекурентна нейронна мережа
CTC	– Connectionist Temporal Classification, декодування послідовності без сегментації символів
H.264	– стандарт стиснення відео (MPEG-4 AVC)
IoU	– Intersection over Union, відношення площі перетину двох рамок до площі їх об'єднання
LSTM	– Long Short-Term Memory, рекурентна мережа з довгою короткочасною пам'яттю
MPS	– Metal Performance Shaders, обчислення на графічному процесорі Apple Silicon
OCR	– Optical Character Recognition, оптичне розпізнавання символів
RANSAC	– Random Sample Consensus, метод стійкого оцінювання параметрів моделі
SFace	– мережа розпізнавання облич (Sigmoid-Constrained Hypersphere Loss)
YOLO	– You Only Look Once, сімейство одностадійних детекторів об'єктів
YOLOv11-LP	– YOLOv11, донавчена на номерних знаках (License Plate)
YuNet	– легкий згортковий детектор облич з OpenCV Zoo

ДСТУ	– національний стандарт України
ПЗ	– програмне забезпечення
трек	– послідовність положень одного об'єкта в кадрах відео з постійним номером
M	– нормована оцінка руху об'єкта після компенсації руху камери
sim	– косинусна схожість векторів ознак облич
px	– піксель
кадр/с	– кадрів за секунду

ВСТУП

Обробка відео є однією з найбільш затребуваних задач комп'ютерного зору: системи відеоспостереження, контролю в'їзду на парковки, автоматичної фіксації порушень правил дорожнього руху та пошуку транспортних засобів поєднують у собі виявлення об'єктів, їх відстеження між кадрами, читання номерних знаків та розпізнавання облич. Сучасні попередньо навчені нейронні мережі дозволяють зібрати такий конвеєр з готових компонентів, а основна інженерна робота полягає в їх правильному поєднанні, постобробці результатів і прийнятті рішень на рівні всього відео, а не окремого кадру.

Мета роботи — розробити елементи програмного забезпечення для обробки відеофільму тривалістю 30–60 с, що містить автомобіль, його номерний знак, обличчя водія біля автомобіля, додаткові об'єкти та один рухомий об'єкт, і з використанням відомих нейронних мереж виконати:

– виділення об'єктів у фільмі та їх розпізнавання (автомобіль, номерний знак, люди, тварина);

– визначення номера автомобіля за допомогою мережі для розпізнавання номерних знаків;

– ідентифікацію водія.

Відповідно до дозволу викладача відеофільм є синтетичним: він програмно зібраний з вільних фотографій і **відрізняється від демонстраційного прикладу** [17] — інший автомобіль (ЗАЗ Sens, вид спереду, на шкільному подвір'ї), власний номерний знак `KA 7429 MB`, інший водій та інше еталонне фото, пасажир з іншого джерела, собака та другий автомобіль як додаткові об'єкти, тривалість 40 с замість 10 с, віртуальна камера «з руки» з наближенням і панорамуванням. Перевагою синтетичного відео є наявність точної еталонної розмітки для кожного кадру, що дозволило оцінити якість роботи кількісно. Звіт оформлено відповідно до ДСТУ 3008:2015 [19].

Для досягнення мети розв'язано такі задачі:

1) сформовано синтетичне відео 40 с (1280×720, 25 кадр/с) з еталонною розміткою;

2) реалізовано виявлення об'єктів мережею YOLO11m та їх відстеження алгоритмом ByteTrack;

3) реалізовано оцінку руху камери оптичним потоком Лукаса–Канаде з RANSAC для коректного визначення рухомого об'єкта;

4) реалізовано розпізнавання номерного знака: детектор YOLOv11-LP, OCR EasyOCR, нормалізацію за шаблоном українського номера та голосування по кадрах;

5) реалізовано ідентифікацію водія мережами YuNet і SFace за окремим еталонним фото;

6) проведено кількісну оцінку результатів, аналіз помилок і запропоновано способи покращення.

1 ПОСТАНОВКА ЗАДАЧІ ТА ВИХІДНІ ДАНІ

1.1 Умова завдання

Згідно із завданням № 3 з дисципліни «Методи обробки мультимедійної інформації» необхідно зняти невеликий фільм тривалістю 30 –60 с, який містить автомобіль, номер автомобіля, обличчя водія біля автомобіля та хоча б один додатковий об'єкт (пасажира, другий автомобіль, рюкзак, домашню тварину тощо). У відео має бути один рухомий об'єкт: машина, людина або тварина. Розроблене ПЗ повинно виділяти вказані об'єкти та розпізнавати їх, визначати номер автомобіля за допомогою відомих мереж і ідентифікувати водія. Дозволено використовувати існуючі технології розпізнавання цифр, облич та об'єктів.

Відповідність вимог завдання вмісту відео наведено в таблиці 1.1.

Таблиця 1.1 – Відповідність вмісту відео вимогам завдання

Вимога завдання	Реалізація у відео
Тривалість 30–60 с	40 с, 1000 кадрів, 25 кадр/с
Автомобіль	ЗАЗ Sens (вид спереду), займає центр кадру
Номер автомобіля	власний знак КА 7429 МВ, ширина в кадрі ≈ 210–290 пікселів
Обличчя водія біля автомобіля	водій стоїть праворуч біля автомобіля, обличчя анфас
Додаткові об'єкти	пасажир, другий (синій) автомобіль, собака
Один рухомий об'єкт	пасажир, що йде зліва направо з 3-ї по 33-тю секунду
Окреме фото водія для ідентифікації	фото того ж водія 2016 р. (інший рік, одяг, освітлення)

1.2 Опис відео

Відео сформоване модулем `scene.py`. Сцена складається з шарів, які накладаються в порядку від дальнього до ближнього: фон з автомобілями, собака, пасажир, водій. Параметри відео наведено в таблиці 1.2, джерела зображень — у таблиці 1.3.

Таблиця 1.2 – Параметри відео

Параметр	Значення
Файл	data/synthetic_scene.mp4
Роздільна здатність	1280×720 пікселів
Частота кадрів	25 кадр/с
Тривалість / кількість кадрів	40 с / 1000 кадрів
Кодек	H.264 (libx264), yuv420p, 8 Мбіт/с
Рух камери	наближення 1,00 → 1,35 → 1,00; панорама; тремтіння ±7 px; поворот ±0,35°
Шумові ефекти	гаусове розмиття σ=0,6; шум сенсора σ=2,2; коливання експозиції ±2,5 %

Таблиця 1.3 – Джерела зображень для сцени

Файл	Призначення	Автор, ліцензія
car_zaz_sens.jpg	фон, головний і другий автомобіль	Вальдимар, CC BY-SA 3.0
driver_in_scene_2012.jpg	водій у відео	UNICEF Ukraine, CC BY 2.0
driver_reference_2016.jpg	еталонне фото водія	Oleg Dubyna, CC BY-SA 2.0
passenger_source.jpg	пасажир (рухомий об'єкт)	Ultralytics, тестове зображення bus.jpg
dog_puppy.jpg	собака (додатковий об'єкт)	Retyi, CC0

Головний автомобіль на вихідному фото має зачорнений номерний знак. На його місце нанесено власний знак `KA 7429 MB`, намальований за пропорціями стандартного українського номерного знака (520×112 мм, синя смуга з прапором і кодом «UA» зліва) згідно з ДСТУ 4278:2019 [18]. Знак переноситься на фото перспективним перетворенням (гомографією), що задається чотирма кутами ділянки знака; потім він затемнюється й трохи розмивається, щоб відповідати освітленню та різкості знімка.

Водія, пасажира та собаку вирізано з їхніх фотографій сегментаційною мережею YOLO11m-seg: полігон маски екземпляра заливається в повній роздільній здатності, маска звужується на 2 пікселі та розмивається, утворюючи м'який альфа-канал. Колірна статистика вирізаних об'єктів частково наближається до фону переносом середнього та дисперсії в просторі L*a*b*, під пасажира й собаку додаються м'які тіні. Пасажир рухається з легким вертикальним «підстрибуванням» кроку.

Віртуальна камера задається ключовими положеннями (центр вікна, наближення) з плавною інтерполяцією smoothstep, до яких додається тремтіння рук як сума синусоїд різних частот. Для кожного кадру будується матриця афінного перетворення «світ → кадр», після чого додаються розмиття, шум сенсора та коливання експозиції. Приклади кадрів наведено на рисунку 1.1.



Рисунок 1.1 – Кадри синтетичного відео: 0 с, 10 с, 18 с і 36 с

1.3 Еталонна розмітка

Оскільки сцена формується програмно, для кожного кадру відомі істинні дані, які зберігаються у файлі `data/synthetic_scene_ground_truth.json`:

– рамки видимої частини кожного об'єкта (головний і другий автомобіль, номерний знак, водій, пасажир, собака) з урахуванням перекриттів шарів і меж кадру;

– частка видимих пікселів об'єкта (видимість від 0 до 1);

– параметри та матриця перетворення камери;

– текст номера (KA7429MB), який із людей є водієм і який об'єкт рухається.

Ця розмітка використовується модулем `evaluation.py` для обчислення метрик якості (розділ 4).

2 ТЕОРЕТИЧНІ ВІДОМОСТІ ТА ВИБІР МОДЕЛЕЙ

2.1 Виявлення об'єктів. YOLO11

YOLO (You Only Look Once) — сімейство одностадійних детекторів, що за один прохід згорткової мережі передбачають рамки, класи та впевненість для всіх об'єктів зображення [1]. YOLO11 (Ultralytics, 2024) складається з магістральної мережі (backbone) з блоками C3k2, шиї (neck) типу FPN/PAN з модулем просторової уваги C2PSA та безякірної (anchor-free) голови, що окремо передбачає класи та координати рамок на трьох масштабах. Модель навчена на наборі COCO (Common Objects in Context) з 80 класів [3], серед яких є всі потрібні в завданні: person, car, truck, bus, dog, cat, backpack, handbag тощо.

Використано варіант YOLO11m (20,1 млн параметрів, mAP 51,5 на COCO val2017) замість YOLO11s з демонстрації: він точніше розрізняє легковики та вантажівки на незвичних ракурсах і стабільніше працює з частково перекритими об'єктами, а швидкості на графічному процесорі Apple M1 (MPS) достатньо.

2.2 Відстеження об'єктів. ByteTrack

Детектор працює з кожним кадром незалежно, тож для аналізу поведінки об'єктів детекції потрібно пов'язати в часі — побудувати треки. ByteTrack [2] прогнозує положення кожного треку фільтром Калмана і зіставляє прогнози з детекціями за перетином рамок (IoU) угорським алгоритмом. Його особливість — двоетапне зіставлення: спершу з упевненими детекціями, а потім з детекціями низької впевненості, які зазвичай відповідають частково перекритим об'єктам. Завдяки цьому об'єкт не «губиться», коли його ненадовго перекриває інший (у нашому відео пасажир проходить перед собакою та номерним знаком).

2.3 Оцінка руху камери

Під час зйомки «з руки» камера наближається, панорамує і тремтить, тому в координатах кадру зміщуються всі об'єкти, навіть нерухомі. Щоб відрізнити власний рух об'єкта від руху камери, оцінюється глобальне перетворення між сусідніми кадрами. На попередньому кадрі детектором Ши–Томасі [11] знаходяться кутові точки фону (ділянки всередині рамок об'єктів маскуються), їх положення на поточному кадрі визначається пірамідальним оптичним потоком Лукаса–Канаде [10, 13]. За парами точок методом RANSAC [12] оцінюється перетворення подібності з чотирма ступенями свободи (масштаб s, кут θ, зсув):

$$x' = s \cdot R(\theta) \cdot x + t \tag{2.1}$$

де x — точка поточного кадру, x′ — відповідна точка попереднього кадру, R(θ) — матриця повороту, t — вектор зсуву. RANSAC багаторазово оцінює модель за випадковими мінімальними підмножинами точок і залишає ту, з якою узгоджується найбільше точок, відкидаючи хибні відповідності. Добуток перетворень від кадру до кадру дає відображення T_t «кадр t → перший кадр», де A_k — перетворення між кадрами k і k − 1:

$$T_t = A_1 \cdot A_2 \cdot \ldots \cdot A_t \tag{2.2}$$

У системі координат першого кадру нерухомі об'єкти залишаються на місці, а рухомі — зміщуються.

2.4 Розпізнавання номерних знаків

Загальна COCO-модель не має класу «номерний знак», тому використано YOLOv11s, донавчену на наборі номерних знаків (`morsetechlab/yolov11-license-plate-detection`, Hugging Face) [4]. Вона знаходить рамку знака, після чого символи читає EasyOCR [5] — двоетапна система, що складається з детектора тексту CRAFT [6] (оцінює для кожного пікселя ймовірність належності до символу та до проміжку між символами) і розпізнавача CRNN [7] (згорткові шари виділяють ознаки,

двонаправлена LSTM моделює послідовність, декодування CTC перетворює її на рядок без потреби в сегментації символів).

Перед OCR знак збільшується втричі бікубічною інтерполяцією, переводиться у відтінки сірого та обробляється CLAHE [16] (адаптивним вирівнюванням гістограми з обмеженням контрасту), що підсилює контури дрібних символів. EasyOCR не знає формату номерів, тому застосовується постобробка за шаблоном українського номера `LLDDDDLL`, де L — одна з 12 літер, що мають однакове написання в кирилиці та латиниці (A, B, C, E, H, I, K, M, O, P, T, X), D — цифра.

2.5 Виявлення та розпізнавання облич. YuNet і SFace

YuNet [8] — мініатюрний згортковий детектор облич (≈ 75 тис. параметрів) з OpenCV Zoo [15], що для кожного обличчя повертає рамку, 5 ключових точок (центри очей, кінчик носа, кутики рота) та впевненість. За ключовими точками обличчя вирівнюється афінним перетворенням до стандартного положення розміром 112×112 пікселів.

SFace [9] — мережа розпізнавання облич, навчена з функцією втрат Sigmoid-Constrained Hypersphere Loss, яка перетворює вирівняне обличчя на 128-вимірний вектор ознак. Обличчя однієї людини дають близькі вектори, різних людей — далекі. Мірою близькості є косинусна схожість:

$$\mathrm{sim}(a,b)=\frac{a\cdot b}{\|a\|\cdot\|b\|} \tag{2.3}$$

де a і b — 128-вимірні вектори ознак двох облич, $\|\cdot\|$ — евклідова норма.

Обличчя вважаються належними одній людині, якщо sim > 0,363 — поріг, рекомендований авторами моделі для косинусної метрики.

2.6 Обґрунтування вибору моделей

Підсумковий перелік використаних моделей і причини їх вибору наведено в таблиці 2.1.

Таблиця 2.1 – Використані моделі та методи

Задача	**Модель / метод**	**Чому обрано**
Виявлення об'єктів	YOLO11m, COCO (80 класів)	сучасний точний одностадійний детектор; усі потрібні класи вже є в COCO
Відстеження	ByteTrack	вбудований в Ultralytics; утримує треки при частковому перекритті
Рух камери	Ши–Томасі + Лукас–Канаде + RANSAC	класичні швидкі методи OpenCV; модель подібності відповідає руху камери
Номерний знак	YOLOv11s-LP (morsetechlab)	відома відкрита мережа, донавчена саме на номерних знаках
Символи номера	EasyOCR (CRAFT + CRNN)	готовий OCR з латиницею та цифрами; не потребує донавчання
Обличчя	YuNet + SFace (OpenCV Zoo)	легкі, точні, працюють у самому OpenCV без додаткових фреймворків
Побудова сцени	YOLO11m-seg	маски екземплярів для вирізання людей і собаки з фото

3 ПРОГРАМНА РЕАЛІЗАЦІЯ

3.1 Середовище та структура проєкту

Програмне забезпечення написане мовою Python 3.14 з використанням бібліотек PyTorch 2.14, Ultralytics 8.4, OpenCV 5.0 [14], EasyOCR 1.7, NumPy, pandas і matplotlib. Нейромережі YOLO виконуються на графічному процесорі Apple M1 через Metal Performance Shaders (MPS). Код організовано як пакет `src/lab3`, кожен модуль якого відповідає за один крок конвеєра (таблиця 3.1). Запуск — скрипт `run_lab3.py` (команди scene, process, all) або ноутбук `notebooks/Lab3_video_processing.ipynb`, який виконує кроки по черзі й показує проміжні результати.

Таблиця 3.1 – Модулі програмного забезпечення

Модуль	Призначення
config.py	шляхи, моделі, порогові значення в одному місці
plate_render.py	малювання номерного знака та перенесення його на фото гомографією
scene.py	генерація синтетичного відео та еталонної розмітки
detection.py	клас ObjectTracker: YOLO11m + ByteTrack
camera_motion.py	клас CameraMotionEstimator: оцінка руху камери
plates.py	клас PlateReader, функції normalize_plate і vote
faces.py	клас FaceIdentifier: YuNet + SFace
analysis.py	рішення по треках: рухомий об'єкт, водій, номер
annotate.py	побудова анотованого відео
evaluation.py	метрики якості за еталонною розміткою
report_figures.py	графіки для звіту
pipeline.py	запуск усього конвеєра, збереження результатів

3.2 Схема конвеєра обробки

Схему конвеєра наведено на рисунку 3.1. Кожен кадр проходить через детектор об'єктів з трекером і оцінювач руху камери; кожен другий кадр (параметр `PROCESS_EVERY = 2`) додатково аналізується на номерний знак і обличчя. Після проходу по всьому відео результати агрегуються по треках.



Рисунок 3.1 – Схема конвеєра обробки відео

Основний цикл обробки реалізовано у функції `process_video` (лістинг 3.1).

Лістинг 3.1 – Фрагмент функції process_video — основний цикл (pipeline.py)

```
while True:
    ok, frame = cap.read()
    if not ok:
        break
    dets = tracker.update(frame)
    T = camera.update(frame, [d.box for d in dets])
    rec = {"frame": f, "t": f / meta["fps"], "dets": dets, "T": T.copy(),
"plates": None, "faces": None}
    if f % C.PROCESS_EVERY == 0:
        rec["plates"] = plates.read(frame)
        rec["faces"] = faces.process(frame)
    records.append(rec)
    f += 1
```

3.3 Виявлення та відстеження об'єктів

Клас `ObjectTracker` (лістинг 3.2) завантажує YOLO11m і залишає лише класи, що стосуються завдання. Виклик `model.track(..., persist=True)` зберігає стан ByteTrack між кадрами, тому кожен об'єкт отримує номер треку, що не змінюється. Результат кадру — список об'єктів `Detection` з номером треку, класом, впевненістю та рамкою. Мінімальна впевненість детекції — 0,35.

Лістинг 3.2 – Метод ObjectTracker.update (detection.py)

```
def update(self, frame: np.ndarray) -> list[Detection]:
    """Обробляє черговий кадр і повертає список відстежуваних об'єктів.


    ``persist=True`` зберігає стан трекера між викликами — саме так
    ByteTrack «пам'ятає» об'єкти з попередніх кадрів.
    """
    r = self.model.track(frame, persist=True, tracker=C.TRACKER_CFG,
conf=self.conf,
                         classes=self.class_ids, device=self.device,
verbose=False)[0]
    if r.boxes.id is None:  # у кадрі немає жодного підтвердженого треку
        return []
    ids = r.boxes.id.int().cpu().tolist()
    cls = r.boxes.cls.int().cpu().tolist()
    conf = r.boxes.conf.cpu().tolist()
    xyxy = r.boxes.xyxy.cpu().numpy()
    return [Detection(t, self.names[c], float(p), tuple(map(float, b)))
            for t, c, p, b in zip(ids, cls, conf, xyxy)]
```

3.4 Компенсація руху камери

Метод `CameraMotionEstimator.update` (лістинг 3.3) реалізує алгоритм з підрозділу 2.3. Маска фону будується з рамок об'єктів, розширених на 8 пікселів, — точки на людях і тваринах не впливають на оцінку. Шукається до 500 кутових точок, оптичний потік обчислюється з вікном 21×21 на чотирьох рівнях піраміди, поріг RANSAC — 1,5 пікселя. Перетворення накопичуються в матрицю 3×3 «поточний кадр → перший кадр».

Лістинг 3.3 – Фрагмент методу CameraMotionEstimator.update (camera_motion.py)

```
if self.prev_gray is not None:
    pts_prev = cv2.goodFeaturesToTrack(self.prev_gray, self.max_corners,
self.quality,
                                       self.min_distance, mask=self.prev_mask)
    step = np.eye(3)
    n_in = 0
    if pts_prev is not None and len(pts_prev) >= 8:
        pts_cur, st, _ = cv2.calcOpticalFlowPyrLK(self.prev_gray, gray, pts_prev,
None,
                                                  winSize=(21, 21), maxLevel=3)
        ok = st.ravel() == 1
        if ok.sum() >= 8:
            # Модель «поточний → попередній»
            A, inl = cv2.estimateAffinePartial2D(pts_cur[ok], pts_prev[ok],
method=cv2.RANSAC,
                                                 ransacReprojThreshold=self.ransac
_thr)
            if A is not None:
                step[:2] = A
                n_in = int(inl.sum())
    self.to_ref = self.to_ref @ step
```

3.5 Розпізнавання номерного знака

Клас `PlateReader` знаходить знак, готує зображення (розширення рамки на 4 % по ширині та 12 % по висоті, збільшення ×3, відтінки сірого, CLAHE) і передає його EasyOCR з алфавітом із великих латинських літер і цифр. Фрагменти тексту впорядковуються зліва направо й об'єднуються в сирий рядок.

Ключова частина — функція `normalize_plate` (лістинг 3.4). Вона перебирає всі вікна по 8 символів сирого рядка і виправляє типові помилки OCR залежно від позиції: на місцях цифр $O \to 0$, $I \to 1$, $B \to 8$, $S \to 5$ тощо, на місцях літер — навпаки. Недопустима для українського номера літера штрафується. Обирається вікно з найменшою кількістю виправлень, а читання з більш ніж двома виправленнями відкидається як ненадійне. Так рядок «UA KA7429MB» (з кодом країни із синьої смуги) перетворюється на KA7429MB, а «7429MB» (половина знака перекрита пасажиром) відкидається.

Лістинг 3.4 – Нормалізація за шаблоном українського номера (plates.py)

```
def normalize_plate(raw: str) -> tuple[str | None, int]:
    """Знаходить у сирому рядку OCR український номер ``LLDDDDLL``.


    Перебирає всі вікна довжиною 8 символів, виправляє символи відповідно
    до позиції (цифра/літера) і повертає варіант з найменшою кількістю
    виправлень. Якщо жодне вікно не вдається привести до шаблону — ``None``.
    """
    s = re.sub(r"[^A-Z0-9]", "", raw.upper())
    best: tuple[str | None, int] = (None, 99)
    for i in range(len(s) - 7):
        win, fixed, n_fix = s[i:i + 8], [], 0
        for pos, ch in enumerate(win):
            if pos in (2, 3, 4, 5):          # позиції цифр
                if not ch.isdigit():
                    ch, n_fix = _TO_DIGIT.get(ch, ch), n_fix + 1
            else:                             # позиції літер
                if ch.isdigit():
                    ch, n_fix = _TO_LETTER.get(ch, ch), n_fix + 1
                if ch not in _LETTERS:
                    n_fix += 5                # недопустима літера — сильний штраф
            fixed.append(ch)
        cand = "".join(fixed)
        if _PATTERN.fullmatch(cand) and n_fix < best[1]:
            best = (cand, n_fix)
    # Більше двох виправлень — читання ненадійне, не довіряємо йому
    return best if best[1] <= 2 else (None, best[1])
```

Результати з різних кадрів об'єднуються функцією `vote` (лістинг 3.5) двома способами: голосуванням за цілий рядок і посимвольним голосуванням, зваженим на впевненість OCR. Посимвольне голосування дає правильний номер навіть тоді, коли кожен кадр має одну помилку в різних позиціях.

Лістинг 3.5 – Фрагмент функції vote — посимвольне голосування (plates.py)

```
valid = [r for r in reads if r.plate]
counts = Counter(r.plate for r in valid)
by_char = None
if valid:
    pos_votes: list[defaultdict] = [defaultdict(float) for _ in range(8)]
    for r in valid:
        for i, ch in enumerate(r.plate):
            pos_votes[i][ch] += max(r.ocr_conf, 0.05)
    by_char = "".join(max(v, key=v.get) for v in pos_votes)
```

3.6 Ідентифікація водія

Клас `FaceIdentifier` (лістинг 3.6) на етапі ініціалізації обчислює вектор SFace для найбільшого обличчя на еталонному фото водія. Для кожного кадру знаходяться всі обличчя з впевненістю YuNet ≥ 0,8, обличчя

менші за 24 пікселі пропускаються, решта вирівнюється і порівнюється з еталоном. Кожне обличчя прив'язується до найменшої рамки `person`, у яку потрапляє його центр, — так схожість накопичується по треках людей.

Лістинг 3.6 – Метод FaceIdentifier.process (faces.py)

```
def process(self, frame: np.ndarray) -> list[FaceMatch]:
    """Знаходить усі обличчя в кадрі та порівнює кожне з еталоном."""
    out = []
    for f in self.detect(frame):
        x, y, w, h = f[:4]
        if w < 24 or h < 24:  # занадто дрібні обличчя SFace розпізнає ненадійно
            continue
        sim = self.similarity(self.embed(frame, f))
        out.append(FaceMatch((float(x), float(y), float(x + w), float(y + h)),
float(f[14]), sim))
    return out
```

3.7 Прийняття рішень по треках

Функція `summarize` модуля `analysis.py` агрегує покадрові результати. Спочатку треки очищуються:

– **об'єднання фрагментів**: два треки одного класу, що майже не співіснують у часі (перекриття не більше 20 % коротшого) і мають однакові медіанні рамки в координатах першого кадру (IoU ≥ 0,6), вважаються одним об'єктом — це виправляє зміну номера треку під час наближення камери;

– **відкидання коротких треків**: треки, коротші за 10 % відео, вважаються шумом;

– **відкидання частин інших об'єктів**: трек, що у ≥ 80 % кадрів лежить усередині рамки довшого треку того ж класу;

– **відкидання крайових треків**: середня впевненість < 0,5 і ≥ 80 % кадрів рамка впирається в край кадру (наприклад, піднята рука водія, що частково потрапила в кадр).

Рухомий об'єкт. Рамки треку переводяться в систему координат першого кадру. Спостереження, де площа рамки менша за 70 % від 90-го перцентиля площ (об'єкт обрізаний краєм кадру або перекритий), відкидаються. Для кожної сторони рамки, що не торкається краю кадру,

обчислюється розмах положень (різниця 90-го та 10-го перцентилів), береться медіана по сторонах і ділиться на характерний розмір об'єкта:

$$M = \frac{median_e\left(P_{90}(e) - P_{10}(e)\right)}{median\left(\sqrt{w \cdot h}\right)} \tag{3.1}$$

де e — сторона рамки (ліва, верхня, права, нижня), P_{90} і P_{10} — 90-й і 10-й перцентилі її положення, w і h — ширина і висота рамки в координатах першого кадру.

Об'єкт вважається рухомим, якщо M > 0,25. Медіана по сторонах робить оцінку стійкою до часткових перекриттів, оскільки перекриття змінює одну-дві сторони рамки, а не всі чотири. Для порівняння обчислюються також дві «наївні» оцінки без компенсації руху камери: зміщення центру рамки від першої до останньої появи (як у демонстрації) і максимальне зміщення центру від першої появи.

Лістинг 3.7 – Фрагмент функції movement_score (analysis.py)

```
refs = [_box_to_ref(b, T) for b, T in samples]
areas = np.array([max(r[2] - r[0], 1) * max(r[3] - r[1], 1) for r in refs])
# Спостереження, де об'єкт видно лише частково (обрізаний краєм кадру
# чи перекритий) мають помітно меншу площу — їх рамка ненадійна.
full = areas >= 0.7 * np.percentile(areas, 90)
edges: list[list[float]] = [[], [], [], []]
sizes = []
for (box, _), ref, ok in zip(samples, refs, full):
    if not ok:
        continue
    sizes.append(np.sqrt(max(ref[2] - ref[0], 1) * max(ref[3] - ref[1], 1)))
    x1, y1, x2, y2 = box
    truncated = [x1 <= m, y1 <= m, x2 >= W - m, y2 >= H - m]
    for i in range(4):
        if not truncated[i]:
            edges[i].append(ref[i])
size = float(np.median(sizes))
ranges = [np.percentile(e, 90) - np.percentile(e, 10) for e in edges if len(e) >=
5]
if not ranges:  # об'єкт увесь час обрізаний краєм кадру — беремо центри
    c = np.array([[(b[0] + b[2]) / 2, (b[1] + b[3]) / 2] for b in (_box_to_ref(b,
T) for b, T in samples)])
    ranges = [float(np.ptp(c[:, 0])), float(np.ptp(c[:, 1]))]
return float(np.median(ranges) / size)
```

Водій — трек класу person, для якого є щонайменше 5 облич і середня схожість з еталоном найбільша; якщо вона не перевищує 0,363, водія не ідентифіковано. **Номер** приписується треку автомобіля, у рамці якого найчастіше знаходився центр знайденого знака.

3.8 Анотоване відео та оцінювання

Модуль `annotate.py` другим проходом по відео наносить на кадри рамки об'єктів з номерами треків, класами та впевненістю, позначки DRIVER (зелена рамка) і MOVING (червона рамка), рамку номерного знака з прочитаним після голосування номером, рамки облич зі значенням схожості та інформаційну панель. Траєкторія рухомого об'єкта зберігається в координатах першого кадру і в кожному кадрі переноситься оберненою матрицею руху камери, тому вона «прив'язана» до сцени, а не до екрана. Написи зроблено латиницею, оскільки шрифти Hershey в OpenCV не підтримують кирилицю. Модуль `evaluation.py` зіставляє треки з еталонними об'єктами за середнім IoU і обчислює метрики, наведені в розділі 4.

4 РЕЗУЛЬТАТИ

Обробка 1000 кадрів на Apple M1 тривала 301 с (3,3 кадр/с) з урахуванням OCR і розпізнавання облич у кожному другому кадрі. Побудова анотованого відео та ілюстрацій займає ще близько 1 хв.

4.1 Виявлені об'єкти

Трекер створив 13 треків. Після очищення об'єднано фрагменти #18→#1, відкинуто частини інших об'єктів #9 та крайові треки #9, #10, #19, #26, решта коротких треків відкинута як шум. Залишилося 5 об'єктів, що точно відповідають вмісту сцени (таблиця 4.1).

Таблиця 4.1 – Підсумкові треки об'єктів

Трек	Клас	Кадрів	Впевн.	Оцінка руху M	Рух	Схожість з водієм (облич)	Роль
#1	car	1000	0,95	0,02	ні	—	автомобіль з номером KA7429MB
#2	person	1000	0,94	0,06	ні	0,707 (498)	водій
#3	dog	994	0,92	0,03	ні	—	додатковий об'єкт (собака)
#4	car	989	0,83	0,02	ні	—	другий автомобіль
#5	person	511	0,89	1,42	так	0,115 (241)	пасажир

Впевненість детектора по кадрах показано на рисунку 4.1. Для автомобіля з номером, водія та собаки вона стабільно перевищує 0,9. Помітні провали на 11–13 с відповідають моменту, коли пасажир проходить перед собакою і передньою частиною автомобіля, а провал пасажира на 24–26 с — моменту, коли він заходить за водія. Другий автомобіль видно лише частково (він обрізаний лівим краєм кадру), тому його впевненість нижча — близько 0,8.

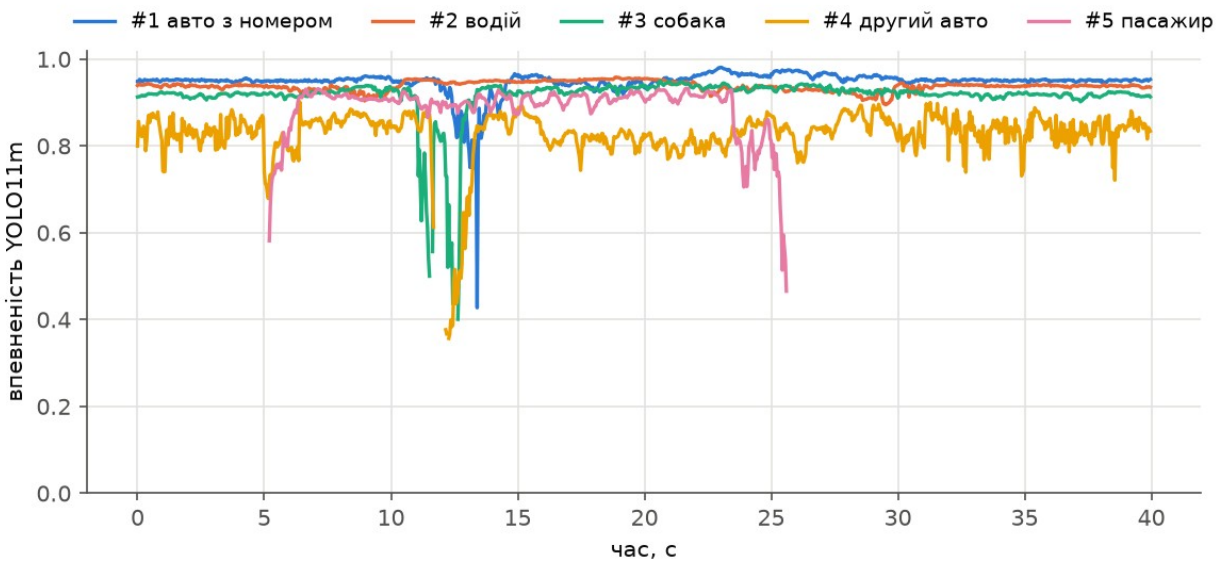


Рисунок 4.1 – Впевненість YOLO11m по кадрах для кожного треку

Приклади кадрів анотованого відео наведено на рисунку 4.2.

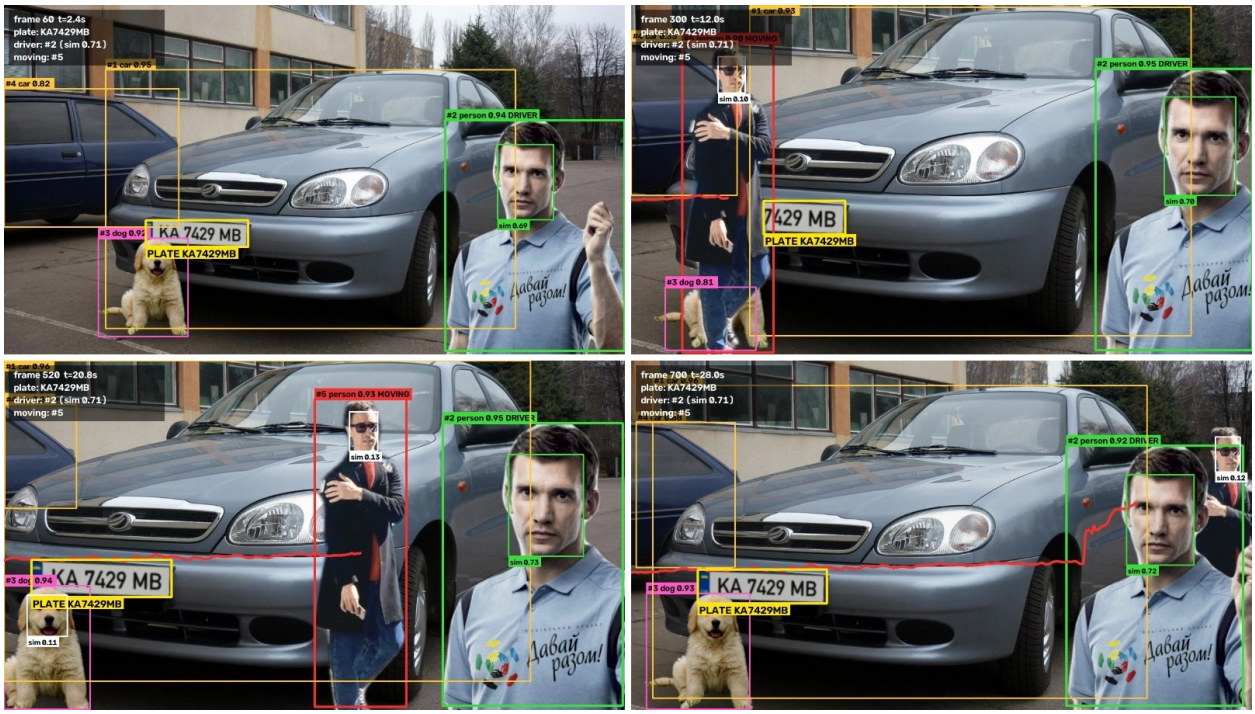


Рисунок 4.2 – Кадри анотованого відео (2,4 с; 12 с; 20,8 с; 28 с)

4.2 Рухомий об'єкт

Оцінки руху для всіх треків трьома способами наведено на рисунку 4.3. З компенсацією руху камери пасажир має оцінку M = 1,42, а всі нерухомі об'єкти — не більше 0,06, тобто рішення має великий запас відносно порогу 0,25.

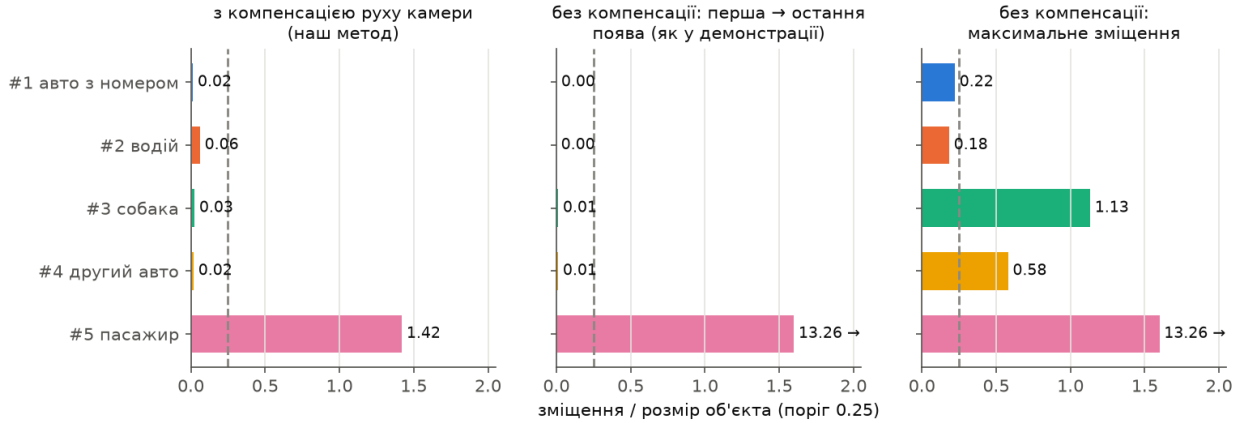


Рисунок 4.3 – Оцінки руху об'єктів з компенсацією і без компенсації руху камери

Без компенсації картина інша. Оцінка «максимальне зміщення» помилково вважає рухомими #3 (додатковий об'єкт (собака), 1,13), #4 (другий автомобіль, 0,58) — під час наближення камери ці об'єкти зміщуються в кадрі більше, ніж на чверть власного розміру. Оцінка з демонстрації (перша → остання поява) у нашому відео дала правильну відповідь лише тому, що камера в кінці повертається у вихідне положення; при іншій траєкторії камери вона б також помилилася. Це підтверджує необхідність компенсації руху камери, про яку згадано в рекомендаціях демонстрації.

Траєкторії центрів усіх треків у координатах першого кадру показано на рисунку 4.4: нерухомі об'єкти утворюють компактні «плями», а пасажир — довгу горизонтальну лінію. Злам наприкінці траєкторії відповідає моменту, коли пасажир заходить за водія і видно лише його голову.



Рисунок 4.4 – Траєкторії об'єктів у системі координат першого кадру

Точність оцінки руху камери перевірено за еталоном (рисунок 4.5). Оцінене наближення практично збігається зі справжнім. Середня похибка перенесення точок кадру в систему першого кадру становить 1,85 пікселя, максимальна — 4,18 пікселя наприкінці відео. Похибка повільно накопичується через добуток перетворень, але залишається на два порядки меншою за розміри об'єктів.

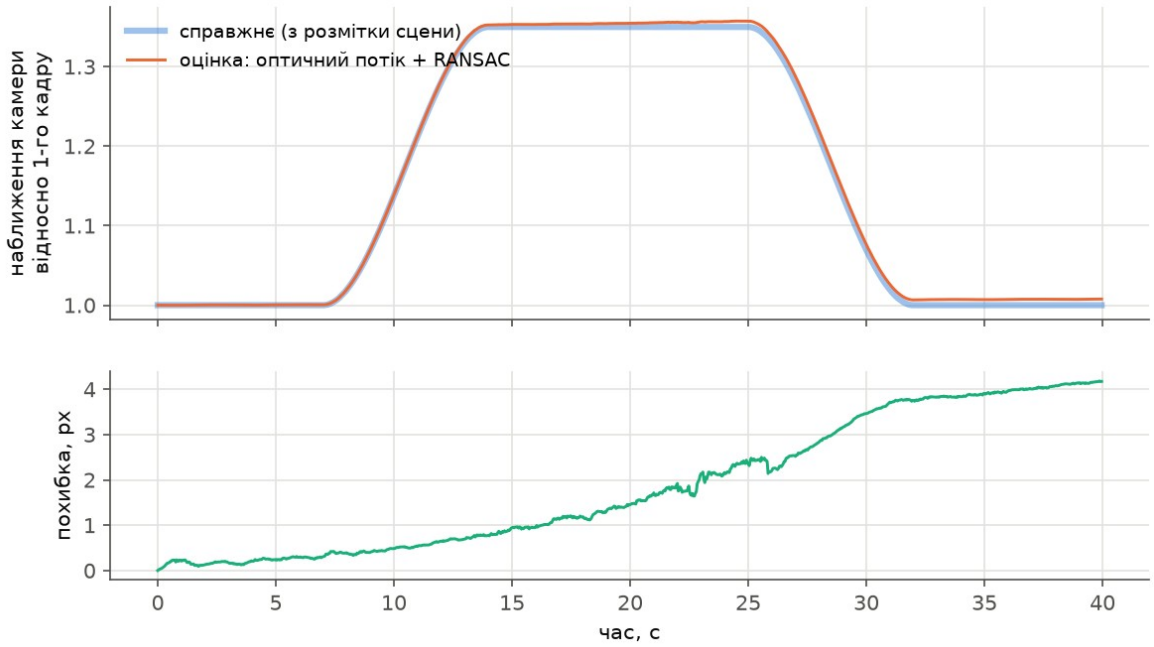


Рисунок 4.5 – Оцінене і справжнє наближення камери та похибка оцінки

4.3 Номер автомобіля

Детектор номерних знаків знайшов знак у 496 проаналізованих кадрах, з них 440 читань відповідали шаблону українського номера. Результати голосування: KA7429MB: 438, KA7429ME: 2. Голосування за рядком дало KA7429MB, посимвольне — KA7429MB. **Номер автомобіля: KA7429MB**, він приписаний треку #1 (автомобіль ЗАЗ Sens), що відповідає еталону.

Приклади підготовки знака до OCR і результати читання наведено на рисунку 4.6. У кадрі 300 пасажир перекриває ліву половину знака: OCR повертає «7429MB», і нормалізація правильно відкидає це читання замість того, щоб видати хибний номер. Код «UA» із синьої смуги, який OCR інколи читає разом із номером, відкидається пошуком вікна з 8 символів.



Рисунок 4.6 – Знайдений номерний знак, підготовлене зображення та результат OCR

4.4 Ідентифікація водія

Схожість облич з еталоном у часі показано на рисунку 4.7. Для водія (трек #2) вона стабільно тримається в межах 0,629–0,746, середнє 0,707 у 498 кадрах, тобто значно вище порогу 0,363. Для пасажира середня схожість 0,115 (максимум 0,726) — значно нижче порогу. **Водія ідентифіковано: трек #2, середня схожість 0,707 > 0,363.**

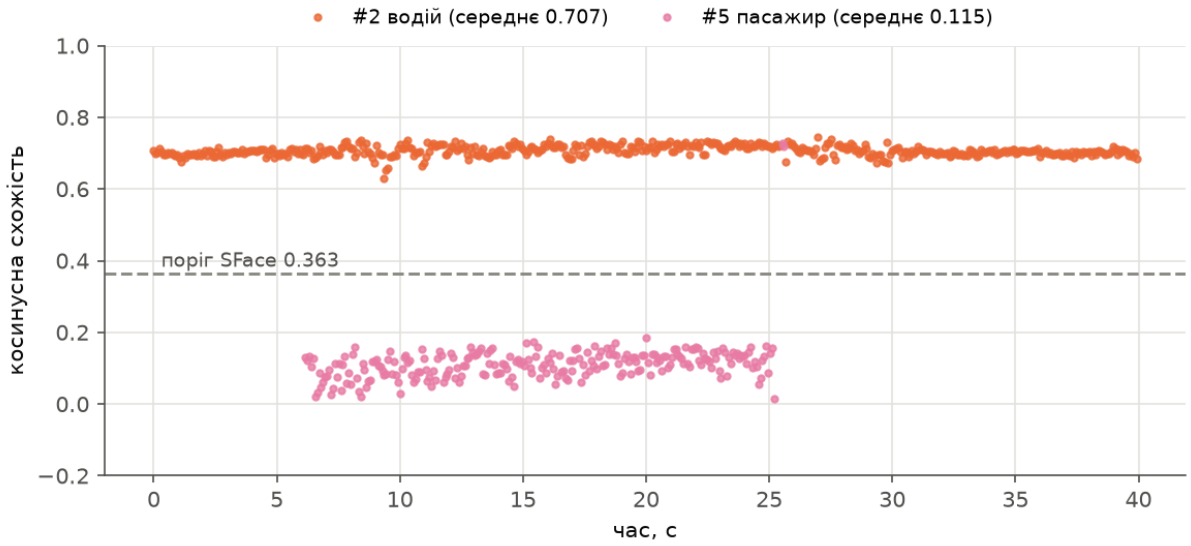


Рисунок 4.7 – Косинусна схожість облич з еталонним фото водія в часі

Вирівняні обличчя, з якими працює SFace, наведено на рисунку 4.8. Хоча еталонне фото зроблене через чотири роки, в іншому одязі та за іншого освітлення, схожість облич водія з еталоном ≈ 0,7. Обличчя пасажира частково закрите сонцезахисними окулярами, і його схожість близька до нуля.



Рисунок 4.8 – Еталонне обличчя та вирівняні обличчя з відео

4.5 Кількісні показники

Метрики якості, обчислені за еталонною розміткою, наведено в таблицях 4.2 і 4.3. Об'єкт вважається знайденим у кадрі, якщо є детекція відповідного класу з IoU ≥ 0,5 з еталонною рамкою; враховуються кадри з видимістю об'єкта ≥ 30 %.

Таблиця 4.2 – Якість виявлення та відстеження об'єктів

Об'єкт	**Кадрів у полі зору**	**Частка знайдених (IoU ≥ 0,5)**	**Середній IoU**	**Номери треків**
автомобіль з номером	1000	1,000	0,968	#1 (1000)
другий автомобіль	636	0,983	0,951	#4 (625)

Об'єкт	Кадрів у полі зору	Частка знайдених (IoU ≥ 0,5)	Середній IoU	Номери треків
водій	1000	0,864	0,960	#2 (864)
пасажир	546	0,967	0,937	#5 (495), #24 (33)
собака	1000	0,994	0,945	#3 (994)

Таблиця 4.3 – Підсумкові показники за пунктами завдання

Показник	Значення
Кадрів з відкритим знаком (видимість ≥ 90 %), проаналізованих на номер	429
Частка кадрів, де знак знайдено	100,0 %
Частка кадрів з читанням правильного формату	100,0 %
Частка кадрів, де номер прочитано точно	100,0 %
Підсумковий номер (голосування) / еталон	KA7429MB / KA7429MB
Ідентифікований водій відповідає еталону	так
Середня схожість: водій / пасажир	0,707 / 0,115
Рухомий об'єкт (з компенсацією) відповідає еталону	так
Те саме без компенсації (максимальне зміщення)	ні — рухомими визнано другий автомобіль, собака, пасажир
Похибка оцінки руху камери: середня / максимальна	1,85 / 4,18 px
Швидкість обробки (Apple M1)	3,3 кадр/с

Нижча частка знайдених кадрів для водія (0,864) пояснюється тим, що під час наближення камери водія видно лише частково: еталонна рамка видимої частини включає підняту руку біля краю кадру, а детектор її не охоплює, тож IoU опускається нижче 0,5. Сам трек водія при цьому не переривається (трек #2 існує у всіх 1000 кадрах). Пасажир розбитий на два треки: після проходу за водієм частина його фігури знову з'являється і отримує новий номер, який відкидається як короткий.

5 АНАЛІЗ ПОМИЛОК ТА СПОСОБИ ПОКРАЩЕННЯ

Незважаючи на правильні підсумкові рішення, на рівні окремих кадрів виявлено такі помилки та обмеження:

1) **Зміна номера треку (ID switch).** Під час швидкого наближення камери ByteTrack присвоїв головному автомобілю новий номер. Виправлено об'єднанням фрагментів у координатах першого кадру; у загальному випадку допоможе трекер з компенсацією руху камери (BoT-SORT з GMC) або ознаками зовнішнього вигляду (re-identification);

2) **Хибні та часткові детекції.** Піднята рука водія біля краю кадру детектувалася як окрема людина з низькою впевненістю. Відфільтровано правилом для крайових треків;

3) **Хибне обличчя.** YuNet в окремих кадрах знаходив «обличчя» на морді собаки (схожість ≈ 0,1). Воно не впливає на результат, бо не потрапляє в рамку жодної людини; можна додатково підвищити поріг впевненості YuNet або перевіряти клас об'єкта, якому належить обличчя;

4) **Перекриття номера.** Коли пасажир або собака перекривають знак, OCR читає лише частину символів. Шаблон номера і голосування по кадрах повністю усувають вплив таких кадрів. EasyOCR не знає формату номерів, тому постобробка обов'язкова; кращий варіант — спеціалізований OCR для номерних знаків (наприклад, fast-plate-ocr) або донавчання CRNN на українських номерах;

5) **Накопичення похибки руху камери.** Похибка зростає приблизно на 0,1 пікселя за секунду; для довших відео варто періодично зіставляти кадр безпосередньо з опорним (ключові кадри) або використовувати гомографію замість моделі подібності при сильних змінах ракурсу;

6) **Окуляри та ракурс обличчя.** Сонцезахисні окуляри пасажира знижують інформативність обличчя. Для водія з окулярами або в профіль схожість може опуститися нижче порогу в окремих кадрах, тому рішення приймається за середнім значенням по треку;

7) **Синтетичність сцени.** Вирізані об'єкти не мають власного руху кінцівок і реалістичних тіней, а освітлення вирівнюється лише статистично. На реальному відео слід очікувати нижчої впевненості детекторів, розмиття в русі та відблисків на номері; рекомендації щодо зйомки — номер ширший за 150 пікселів, обличчя водія анфас, камера відносно нерухома;

8) **Ліцензії.** Модель номерних знаків і YOLO11 поширюються за ліцензією AGPL-3.0: для навчання використання дозволене, для комерційного продукту потрібна інша ліцензія.

ВИСНОВКИ

У роботі розроблено програмне забезпечення для обробки відео, яке з використанням відомих попередньо навчених нейронних мереж виконує всі пункти завдання. Для перевірки сформовано синтетичний відеофільм тривалістю 40 с (1000 кадрів, 1280×720), що відрізняється від демонстраційного прикладу автомобілем, номером, водієм, додатковими об'єктами, тривалістю та рухом камери, і для якого є точна еталонна розмітка.

Мережа YOLO11m разом з трекером ByteTrack виділила й розпізнала всі 5 об'єктів сцени: автомобіль з номером, другий автомобіль, водія, пасажира та собаку; частка кадрів зі знайденим об'єктом становить 0,86–1,00 при середньому IoU 0,94–0,97.

Мережа YOLOv11-LP знайшла номерний знак у 100 % кадрів, де він відкритий, а EasyOCR з постобробкою за шаблоном українського номера прочитав його точно в 100 % таких кадрів. Голосування по кадрах дало номер **KA7429MB**, що збігається з еталоном.

Мережі YuNet і SFace за окремим еталонним фото ідентифікували водія (трек #2): середня косинусна схожість 0,707 проти 0,115 для пасажира при порозі 0,363.

Єдиний рухомий об'єкт — пасажир — визначено з оцінкою 1,42 при порозі 0,25. Показано, що без компенсації руху камери нерухомі об'єкти (собака, другий автомобіль) помилково визнаються рухомими; запропонований метод на основі оптичного потоку та RANSAC оцінює рух камери з середньою похибкою 1,85 пікселя.

Головні інженерні рішення, що забезпечили правильний результат: прийняття рішень по треку, а не по окремому кадру; постобробка OCR за шаблоном номера з голосуванням; компенсація руху камери; очищення треків (об'єднання фрагментів, відкидання хибних і часткових детекцій). Напрями подальшого покращення — спеціалізований OCR для номерів,

трекер з компенсацією руху камери та ознаками зовнішнього вигляду, перевірка на реальному відео.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Jocher G., Qiu J. Ultralytics YOLO11 : software. Version 11.0.0. 2024. URL: https://github.com/ultralytics/ultralytics (дата звернення: 22.10.2026).

2. Zhang Y., Sun P., Jiang Y. et al. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. Computer Vision – ECCV 2022. Cham : Springer, 2022. P. 1–21.

3. Lin T.-Y., Maire M., Belongie S. et al. Microsoft COCO: Common Objects in Context. Computer Vision – ECCV 2014. Cham : Springer, 2014. P. 740–755.

4. morsetechlab. YOLOv11 License Plate Detection : model. Hugging Face, 2024. URL: https://huggingface.co/morsetechlab/yolov11-license-plate-detection (дата звернення: 22.10.2026).

5. JaidedAI. EasyOCR : software. Version 1.7.2. URL: https://github.com/JaidedAI/EasyOCR (дата звернення: 22.10.2026).

6. Baek Y., Lee B., Han D. et al. Character Region Awareness for Text Detection. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019. P. 9365–9374.

7. Shi B., Bai X., Yao C. An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition. IEEE Transactions on Pattern Analysis and Machine Intelligence. 2017. Vol. 39, No. 11. P. 2298–2304.

8. Wu W., Peng H., Yu S. YuNet: A Tiny Millisecond-level Face Detector. Machine Intelligence Research. 2023. Vol. 20, No. 5. P. 656–665.

9. Zhong Y., Deng W., Hu J. et al. SFace: Sigmoid-Constrained Hypersphere Loss for Robust Face Recognition. IEEE Transactions on Image Processing. 2021. Vol. 30. P. 2587–2598.

10. Lucas B. D., Kanade T. An Iterative Image Registration Technique with an Application to Stereo Vision. Proceedings of the 7th International Joint Conference on Artificial Intelligence. 1981. P. 674–679.

11. Shi J., Tomasi C. Good Features to Track. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 1994. P. 593–600.

12. Fischler M. A., Bolles R. C. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. Communications of the ACM. 1981. Vol. 24, No. 6. P. 381–395.

13. Bouguet J.-Y. Pyramidal Implementation of the Lucas Kanade Feature Tracker. Description of the Algorithm. Intel Corporation, Microprocessor Research Labs, 2001. 9 p.

14. Bradski G. The OpenCV Library. Dr. Dobb's Journal of Software Tools. 2000. Vol. 25, No. 11. P. 120–125.

15. OpenCV Zoo: models for OpenCV DNN : repository. URL: https://github.com/opencv/opencv_zoo (дата звернення: 22.10.2026).

16. Zuiderveld K. Contrast Limited Adaptive Histogram Equalization. Graphics Gems IV. San Diego : Academic Press, 1994. P. 474–485.

17. Демонстрація до завдання № 3. Обробка відео: об'єкти, номер автомобіля, ідентифікація водія : навчальні матеріали з дисципліни «Методи обробки мультимедійної інформації». Київ, 2026. 6 с.

18. ДСТУ 4278:2019. Дорожній транспорт. Знаки номерні транспортних засобів. Загальні вимоги. Правила застосування. Київ : ДП «УкрНДНЦ», 2019.

19. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. Київ : ДП «УкрНДНЦ», 2016. 26 с.

ДОДАТОК А

Вихідний код основних модулів

Повний код проєкту (включно з генератором сцени `scene.py`, рендером номера `plate_render.py`, модулями `annotate.py`, `evaluation.py`, `report_figures.py` і ноутбуком) знаходиться в репозиторії роботи. Нижче наведено модулі, що реалізують обробку відео.

Лістинг А.1 – Модуль config.py

```
"""Конфігурація лабораторної роботи №3 «Обробка відео».


Усі шляхи, назви моделей і порогові значення зібрано в одному місці, щоб
модулі конвеєра не містили «магічних чисел», а звіт міг посилатися на
конкретні параметри.
"""
from pathlib import Path


# ---------------------------------------------------------------------------
# Шляхи проєкту
# ---------------------------------------------------------------------------
ROOT = Path(__file__).resolve().parents[2]  # корінь репозиторію
ASSETS_DIR = ROOT / "assets"                # вихідні фото для синтетичної сцени
MODELS_DIR = ROOT / "models"                # ваги нейромереж
DATA_DIR = ROOT / "data"                    # згенероване відео та результати
FIGURES_DIR = ROOT / "report" / "figures"   # ілюстрації для звіту


SCENE_VIDEO = DATA_DIR / "synthetic_scene.mp4"            # вхідне відео (30–60 с)
SCENE_GT = DATA_DIR / "synthetic_scene_ground_truth.json"  # еталонна розмітка сцени
ANNOTATED_VIDEO = DATA_DIR / "synthetic_scene_annotated.mp4"
RESULTS_JSON = DATA_DIR / "results.json"
FRAMES_CSV = DATA_DIR / "frame_log.csv"


# Еталонне фото водія — окреме фото (інший рік, одяг, освітлення), не кадр із відео.
DRIVER_REFERENCE = ASSETS_DIR / "driver_reference_2016.jpg"


# ---------------------------------------------------------------------------
# Моделі (усі — відомі попередньо навчені мережі)
# ---------------------------------------------------------------------------
COCO_MODEL = MODELS_DIR / "yolo11m.pt"                     # YOLO11m, COCO (80 класів)
SEG_MODEL = MODELS_DIR / "yolo11m-seg.pt"                  # YOLO11m-seg — лише для
побудови сцени
PLATE_MODEL = MODELS_DIR / "license-plate-finetune-v1s.pt"  # YOLOv11s, донавчена на
номерних знаках
YUNET_MODEL = MODELS_DIR / "face_detection_yunet_2023mar.onnx"   # детектор облич YuNet
SFACE_MODEL = MODELS_DIR / "face_recognition_sface_2021dec.onnx"  # дескриптор облич SFace


# Класи COCO, які нас цікавлять за умовою завдання (автомобілі, люди, тварини, рюкзаки…)
TARGET_CLASSES = {
    "person", "car", "truck", "bus", "motorcycle", "bicycle",
    "dog", "cat", "backpack", "handbag", "suitcase",
}


# ---------------------------------------------------------------------------
# Параметри обробки
# ---------------------------------------------------------------------------
DET_CONF = 0.35          # мінімальна впевненість детектора об'єктів
TRACKER_CFG = "bytetrack.yaml"  # ByteTrack, вбудований в Ultralytics
PROCESS_EVERY = 2        # номер і обличчя аналізуємо в кожному 2-му кадрі
```

```
PLATE_CONF = 0.40        # мінімальна впевненість детектора номерів
PLATE_UPSCALE = 3        # у скільки разів збільшуємо вирізаний номер перед OCR
# Шаблон українського номера: 2 літери, 4 цифри, 2 літери (лише латинські
# «двійники» кириличних літер, дозволені ДСТУ 4278:2019: A B C E H I K M O P T X).
UA_PLATE_LETTERS = "ABCEHIKMOPTX"
UA_PLATE_REGEX = rf"[{UA_PLATE_LETTERS}]{{2}}\d{{4}}[{UA_PLATE_LETTERS}]{{2}}"


FACE_SCORE = 0.80        # поріг впевненості YuNet
SFACE_COSINE_THRESHOLD = 0.363  # поріг косинусної схожості SFace (рекомендація авторів
моделі)
MIN_FACE_SAMPLES = 5     # мінімум облич на треку, щоб робити висновок про особу


MIN_TRACK_FRACTION = 0.10  # треки, коротші за 10 % відео, вважаємо шумом
MOVING_THRESHOLD = 0.25    # зміщення > 25 % розміру об'єкта (після компенсації руху
камери)
BORDER_MARGIN = 4          # рамки, що торкаються краю кадру (обрізані), не беремо для
оцінки руху


DEVICE = "mps"  # Apple Silicon GPU; на інших машинах — "cuda" або "cpu"
```

Лістинг А.2 – Модуль detection.py

```
"""Виявлення та відстеження об'єктів: YOLO11 (COCO) + ByteTrack.


YOLO11 (You Only Look Once, v11, Ultralytics) за один прохід мережі
повертає рамки, класи та впевненість для 80 класів набору COCO. Ми
залишаємо лише класи, що стосуються завдання (люди, автомобілі, тварини,
рюкзаки/сумки).


ByteTrack пов'язує детекції сусідніх кадрів в «треки»: кожен об'єкт
отримує номер, що зберігається між кадрами. Це дозволяє рахувати
статистику по об'єкту (скільки кадрів видно, як змістився, чиє обличчя).
Особливість ByteTrack — він використовує і детекції з низькою
впевненістю для продовження вже існуючих треків, тому об'єкт не
«губиться», коли його частково перекриває інший.
"""
from __future__ import annotations


from dataclasses import dataclass


import numpy as np
from ultralytics import YOLO


from . import config as C



@dataclass
class Detection:
    """Один об'єкт в одному кадрі."""
    track_id: int
    cls: str
    conf: float
    box: tuple[float, float, float, float]  # x1, y1, x2, y2 у пікселях кадру


    @property
    def center(self) -> tuple[float, float]:
        x1, y1, x2, y2 = self.box
        return (x1 + x2) / 2, (y1 + y2) / 2



class ObjectTracker:
    """Обгортка над ``YOLO.track`` з фільтром класів і зручним форматом виходу."""
```

```
    def __init__(self, model_path=C.COCO_MODEL, conf: float = C.DET_CONF,
                 device: str = C.DEVICE, classes=C.TARGET_CLASSES):
        self.model = YOLO(str(model_path))
        self.names = self.model.names  # id класу → назва
        self.class_ids = [i for i, n in self.names.items() if n in classes]
        self.conf = conf
        self.device = device


    def update(self, frame: np.ndarray) -> list[Detection]:
        """Обробляє черговий кадр і повертає список відстежуваних об'єктів.


        ``persist=True`` зберігає стан трекера між викликами — саме так
        ByteTrack «пам'ятає» об'єкти з попередніх кадрів.
        """
        r = self.model.track(frame, persist=True, tracker=C.TRACKER_CFG, conf=self.conf,
                             classes=self.class_ids, device=self.device, verbose=False)[0]
        if r.boxes.id is None:  # у кадрі немає жодного підтвердженого треку
            return []
        ids = r.boxes.id.int().cpu().tolist()
        cls = r.boxes.cls.int().cpu().tolist()
        conf = r.boxes.conf.cpu().tolist()
        xyxy = r.boxes.xyxy.cpu().numpy()
        return [Detection(t, self.names[c], float(p), tuple(map(float, b)))
                for t, c, p, b in zip(ids, cls, conf, xyxy)]
```

Лістинг А.3 – Модуль camera_motion.py

```
"""Оцінка руху камери (глобального руху кадру).


Відео знято «з руки»: камера наближається, панорамує й тремтить. Через це
в координатах кадру зміщуються *всі* об'єкти, навіть нерухомі. Щоб
визначити, який об'єкт рухається насправді, потрібно відокремити рух
камери від руху об'єкта.


Алгоритм для кожної пари сусідніх кадрів:


1. На попередньому кадрі шукаємо «добрі для відстеження» кутові точки
   (детектор Ши–Томасі, ``cv2.goodFeaturesToTrack``) — лише на фоні:
   ділянки всередині рамок об'єктів маскуються, бо об'єкти можуть рухатися.
2. Знаходимо ці точки на поточному кадрі пірамідальним оптичним потоком
   Лукаса–Канаде (``cv2.calcOpticalFlowPyrLK``).
3. За парами точок оцінюємо перетворення подібності (зсув + поворот +
   масштаб, 4 ступені свободи) методом RANSAC
   (``cv2.estimateAffinePartial2D``) — він відкидає хибні відповідності.
4. Перемножуючи перетворення від кадру до кадру, отримуємо відображення
   «поточний кадр → перший кадр». У цій спільній системі координат
   нерухомі об'єкти стоять на місці, а рухомі — ні.
"""
from __future__ import annotations


import cv2
import numpy as np



class CameraMotionEstimator:
    """Накопичує перетворення «кадр t → кадр 0» (3×3 матриці)."""


    def __init__(self, max_corners: int = 500, quality: float = 0.01, min_distance: int =
10,
                 ransac_thr: float = 1.5):
        self.max_corners = max_corners
        self.quality = quality
        self.min_distance = min_distance
        self.ransac_thr = ransac_thr
        self.prev_gray: np.ndarray | None = None
        self.prev_mask: np.ndarray | None = None
```

```
        self.to_ref = np.eye(3)            # поточний кадр → перший кадр
        self.history: list[np.ndarray] = []  # to_ref для кожного кадру
        self.inliers: list[int] = []         # скільки точок підтвердили модель


    @staticmethod
    def background_mask(shape, boxes, pad: int = 8) -> np.ndarray:
        """Маска фону: 255 скрізь, крім (розширених на ``pad``) рамок об'єктів."""
        m = np.full(shape[:2], 255, np.uint8)
        for x1, y1, x2, y2 in boxes:
            m[max(int(y1) - pad, 0):int(y2) + pad, max(int(x1) - pad, 0):int(x2) + pad] = 0
        return m


    def update(self, frame: np.ndarray, object_boxes) -> np.ndarray:
        """Додає кадр і повертає матрицю 3×3 «цей кадр → перший кадр»."""
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        mask = self.background_mask(gray.shape, object_boxes)


        if self.prev_gray is not None:
            pts_prev = cv2.goodFeaturesToTrack(self.prev_gray, self.max_corners,
self.quality,
                                               self.min_distance, mask=self.prev_mask)
            step = np.eye(3)
            n_in = 0
            if pts_prev is not None and len(pts_prev) >= 8:
                pts_cur, st, _ = cv2.calcOpticalFlowPyrLK(self.prev_gray, gray, pts_prev,
None,
                                                          winSize=(21, 21), maxLevel=3)
                ok = st.ravel() == 1
                if ok.sum() >= 8:
                    # Модель «поточний → попередній»
                    A, inl = cv2.estimateAffinePartial2D(pts_cur[ok], pts_prev[ok],
method=cv2.RANSAC,
                                                         ransacReprojThreshold=self.ransac_
thr)
                    if A is not None:
                        step[:2] = A
                        n_in = int(inl.sum())
            self.to_ref = self.to_ref @ step
            self.inliers.append(n_in)
        else:
            self.inliers.append(0)


        self.prev_gray, self.prev_mask = gray, mask
        self.history.append(self.to_ref.copy())
        return self.to_ref


    @staticmethod
    def apply(T: np.ndarray, pts) -> np.ndarray:
        """Переводить точки (N×2) з координат кадру в координати першого кадру."""
        p = np.asarray(pts, np.float64).reshape(-1, 2)
        ph = np.hstack([p, np.ones((len(p), 1))]) @ T.T
        return ph[:, :2] / ph[:, 2:3]
```

Лістинг А.4 – Модуль plates.py

```
"""Розпізнавання номерного знака: детектор YOLOv11-LP + EasyOCR + постобробка.


Конвеєр для одного кадру:


1. **Пошук знака.** YOLOv11s, донавчена на номерних знаках
   (``morsetechlab/yolov11-license-plate-detection``, Hugging Face), повертає
   рамку знака. Загальна COCO-модель такого класу не має.
2. **Підготовка.** Рамку трохи розширюємо, вирізаємо знак, збільшуємо в
   ``PLATE_UPSCALE`` разів (бікубічна інтерполяція) і переводимо у відтінки
   сірого з вирівнюванням контрасту CLAHE — так OCR краще бачить дрібні символи.
3. **OCR.** EasyOCR = детектор тексту CRAFT + розпізнавач CRNN
   (згорткова рекурентна мережа з CTC-декодуванням). Обмежуємо алфавіт
   латинськими великими літерами й цифрами.
```

```
4. **Постобробка.** OCR «не знає» формату номерів: прапорець і «UA» на
   синій смузі, сплутування O/0, I/1, B/8 тощо. Тому шукаємо в рядку вікно
   з 8 символів, яке після мінімальної кількості виправлень (з урахуванням
   позиції: літери — на місцях 1–2 і 7–8, цифри — на 3–6) відповідає
   шаблону українського номера ``LLDDDDLL``.
5. **Голосування.** Окремі кадри можуть читатися з помилками, тому
   результати з усіх кадрів об'єднуються: голосуванням за цілий рядок і
   посимвольним голосуванням (зважено на впевненість OCR).
"""
from __future__ import annotations


import re
from collections import Counter, defaultdict
from dataclasses import dataclass


import cv2
import easyocr
import numpy as np
from ultralytics import YOLO


from . import config as C


# Типові помилки OCR: що «насправді» означає символ на позиції цифри / літери
_TO_DIGIT = {"O": "0", "D": "0", "Q": "0", "U": "0", "I": "1", "L": "1", "J": "1", "T":
"7",
             "Z": "2", "S": "5", "B": "8", "G": "6", "A": "4"}
_TO_LETTER = {"0": "O", "1": "I", "8": "B", "4": "A", "6": "G", "7": "T", "5": "S", "2":
"Z"}
_PATTERN = re.compile(C.UA_PLATE_REGEX)
_LETTERS = set(C.UA_PLATE_LETTERS)



@dataclass
class PlateRead:
    """Результат обробки одного знака в одному кадрі."""
    box: tuple[float, float, float, float]
    det_conf: float
    raw_text: str            # що повернув OCR (усі фрагменти зліва направо)
    ocr_conf: float          # середня впевненість OCR
    plate: str | None        # нормалізований номер або None, якщо шаблон не знайдено
    corrections: int = 0     # скільки символів довелося виправити



def normalize_plate(raw: str) -> tuple[str | None, int]:
    """Знаходить у сирому рядку OCR український номер ``LLDDDDLL``.


    Перебирає всі вікна довжиною 8 символів, виправляє символи відповідно
    до позиції (цифра/літера) і повертає варіант з найменшою кількістю
    виправлень. Якщо жодне вікно не вдається привести до шаблону — ``None``.
    """
    s = re.sub(r"[^A-Z0-9]", "", raw.upper())
    best: tuple[str | None, int] = (None, 99)
    for i in range(len(s) - 7):
        win, fixed, n_fix = s[i:i + 8], [], 0
        for pos, ch in enumerate(win):
            if pos in (2, 3, 4, 5):          # позиції цифр
                if not ch.isdigit():
                    ch, n_fix = _TO_DIGIT.get(ch, ch), n_fix + 1
            else:                             # позиції літер
                if ch.isdigit():
                    ch, n_fix = _TO_LETTER.get(ch, ch), n_fix + 1
                if ch not in _LETTERS:
                    n_fix += 5                # недопустима літера — сильний штраф
            fixed.append(ch)
        cand = "".join(fixed)
        if _PATTERN.fullmatch(cand) and n_fix < best[1]:
            best = (cand, n_fix)
    # Більше двох виправлень — читання ненадійне, не довіряємо йому
    return best if best[1] <= 2 else (None, best[1])
```

```
class PlateReader:
    """Детектор номерних знаків + OCR."""

    def __init__(self, model_path=C.PLATE_MODEL, conf: float = C.PLATE_CONF,
                 device: str = C.DEVICE, upscale: int = C.PLATE_UPSCALE):
        self.det = YOLO(str(model_path))
        self.ocr = easyocr.Reader(["en"], gpu=False, verbose=False)
        self.conf, self.device, self.upscale = conf, device, upscale
        self.clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(4, 8))

    def detect(self, frame: np.ndarray) -> list[tuple[tuple[float, ...], float]]:
        """Повертає список (рамка, впевненість) знайдених номерних знаків."""
        r = self.det(frame, conf=self.conf, device=self.device, verbose=False)[0]
        return [(tuple(map(float, b)), float(c))
                for b, c in zip(r.boxes.xyxy.cpu().numpy(), r.boxes.conf.cpu().numpy())]

    def preprocess(self, frame: np.ndarray, box) -> np.ndarray:
        """Вирізає знак з невеликим запасом, збільшує, сірий + CLAHE."""
        x1, y1, x2, y2 = box
        pw, ph = 0.04 * (x2 - x1), 0.12 * (y2 - y1)
        H, W = frame.shape[:2]
        crop = frame[max(int(y1 - ph), 0):min(int(y2 + ph), H), max(int(x1 - pw),
0):min(int(x2 + pw), W)]
        crop = cv2.resize(crop, None, fx=self.upscale, fy=self.upscale,
interpolation=cv2.INTER_CUBIC)
        gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
        return self.clahe.apply(gray)

    def read(self, frame: np.ndarray) -> list[PlateRead]:
        """Знаходить і читає всі номери в кадрі."""
        out = []
        for box, det_conf in self.detect(frame):
            img = self.preprocess(frame, box)
            res = self.ocr.readtext(img, allowlist="ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789",
                                    paragraph=False, detail=1)
            res.sort(key=lambda r: min(p[0] for p in r[0]))  # фрагменти зліва направо
            raw = " ".join(r[1] for r in res)
            conf = float(np.mean([r[2] for r in res])) if res else 0.0
            plate, n_fix = normalize_plate(raw)
            out.append(PlateRead(box, det_conf, raw, conf, plate, n_fix))
        return out



def vote(reads: list[PlateRead]) -> dict:
    """Об'єднує читання з різних кадрів.

    * ``by_string`` — найчастіший нормалізований рядок (голоси = кількість кадрів);
    * ``by_char``   — посимвольне голосування, зважене на впевненість OCR:
      навіть якщо кожен кадр має одну помилку в різних місцях, разом вони
      дають правильний номер.
    """
    valid = [r for r in reads if r.plate]
    counts = Counter(r.plate for r in valid)
    by_char = None
    if valid:
        pos_votes: list[defaultdict] = [defaultdict(float) for _ in range(8)]
        for r in valid:
            for i, ch in enumerate(r.plate):
                pos_votes[i][ch] += max(r.ocr_conf, 0.05)
        by_char = "".join(max(v, key=v.get) for v in pos_votes)
    return {
        "frames_with_plate": len(reads),
        "valid_reads": len(valid),
        "counts": dict(counts.most_common()),
        "by_string": counts.most_common(1)[0][0] if counts else None,
        "by_char": by_char,
    }
```

Лістинг А.5 – Модуль faces.py

```
"""Виявлення облич і ідентифікація водія: YuNet + SFace (OpenCV Zoo).


* **YuNet** — легкий згортковий детектор облич (≈75 тис. параметрів).
  Повертає рамку обличчя, 5 ключових точок (очі, ніс, кутики рота) і
  впевненість.
* **SFace** — мережа розпізнавання облич (MobileFaceNet-подібна,
  навчена з функцією втрат SFace на MS1M). Обличчя спершу *вирівнюється*
  за 5 ключовими точками (``alignCrop`` — афінне перетворення до 112×112),
  а потім перетворюється на вектор-ознаку з 128 чисел.


Дві фотографії вважаються однією людиною, якщо косинусна схожість їх
векторів перевищує 0.363 (поріг, рекомендований авторами моделі).
Порівнюємо кожне обличчя з відео з **окремим** еталонним фото водія.
"""
from __future__ import annotations


from dataclasses import dataclass
from pathlib import Path


import cv2
import numpy as np


from . import config as C



@dataclass
class FaceMatch:
    """Обличчя, знайдене в кадрі, та його схожість з еталоном водія."""
    box: tuple[float, float, float, float]  # x1, y1, x2, y2
    score: float                            # впевненість YuNet
    similarity: float                       # косинусна схожість з еталоном


    @property
    def center(self) -> tuple[float, float]:
        x1, y1, x2, y2 = self.box
        return (x1 + x2) / 2, (y1 + y2) / 2



class FaceIdentifier:
    def __init__(self, yunet=C.YUNET_MODEL, sface=C.SFACE_MODEL, score_thr: float =
C.FACE_SCORE):
        self.detector = cv2.FaceDetectorYN.create(str(yunet), "", (320, 320), score_thr,
0.3, 5000)
        self.recognizer = cv2.FaceRecognizerSF.create(str(sface), "")
        self.score_thr = score_thr
        self.reference: np.ndarray | None = None
        self.reference_face: np.ndarray | None = None  # вирівняне еталонне обличчя (для
звіту)


    def detect(self, img: np.ndarray, score_thr: float | None = None) -> np.ndarray:
        """Повертає масив N×15: x, y, w, h, 5 пар координат точок, впевненість."""
        self.detector.setInputSize((img.shape[1], img.shape[0]))
        self.detector.setScoreThreshold(score_thr or self.score_thr)
        _, faces = self.detector.detect(img)
        return np.zeros((0, 15), np.float32) if faces is None else faces


    def embed(self, img: np.ndarray, face: np.ndarray) -> np.ndarray:
        """Вирівнює обличчя за ключовими точками і рахує 128-вимірний вектор SFace."""
        aligned = self.recognizer.alignCrop(img, face)
        return self.recognizer.feature(aligned)
```

```
    def set_reference(self, path: Path) -> None:
        """Завантажує еталонне фото водія; береться найбільше обличчя на фото."""
        img = cv2.imread(str(path))
        faces = self.detect(img, score_thr=0.5)
        if len(faces) == 0:
            raise RuntimeError(f"На еталонному фото {path} не знайдено обличчя")
        face = max(faces, key=lambda f: f[2] * f[3])
        self.reference_face = self.recognizer.alignCrop(img, face)
        self.reference = self.recognizer.feature(self.reference_face)


    def similarity(self, feat: np.ndarray) -> float:
        """Косинусна схожість вектора з еталоном водія."""
        return float(self.recognizer.match(self.reference, feat,
cv2.FaceRecognizerSF_FR_COSINE))


    def process(self, frame: np.ndarray) -> list[FaceMatch]:
        """Знаходить усі обличчя в кадрі та порівнює кожне з еталоном."""
        out = []
        for f in self.detect(frame):
            x, y, w, h = f[:4]
            if w < 24 or h < 24:  # занадто дрібні обличчя SFace розпізнає ненадійно
                continue
            sim = self.similarity(self.embed(frame, f))
            out.append(FaceMatch((float(x), float(y), float(x + w), float(y + h)),
float(f[14]), sim))
        return out
```

Лістинг А.6 – Модуль analysis.py

```
"""Аналіз результатів по треках: рухомий об'єкт, водій, номер автомобіля.


Покадрові результати детекторів «шумні»: об'єкт може бути не знайдений в
окремому кадрі, обличчя — повернуте, номер — прочитаний з помилкою.
Тому всі рішення приймаються **по треку** (по всій історії об'єкта), а не
по одному кадру.


**Рухомий об'єкт.** Положення рамок переводяться в систему координат
першого кадру (компенсація руху камери, див. :mod:`camera_motion`). Для
кожної з чотирьох сторін рамки рахуємо розмах положень (90-й мінус 10-й
перцентиль), беремо медіану по сторонах і ділимо на характерний розмір
об'єкта √(w·h). Об'єкт рухається, якщо ця величина > 0.25. Медіана по
сторонах робить оцінку стійкою до часткових перекриттів (змінюється одна
сторона рамки, а не всі), а сторони, що впираються в край кадру
(обрізаний об'єкт), не враховуються.


Для порівняння рахується і «наївна» оцінка з демонстрації — зміщення
центру рамки від першої до останньої появи у координатах кадру, без
компенсації руху камери.


Перед аналізом треки «очищуються»: фрагменти одного нерухомого об'єкта,
що отримали різні номери (ID switch під час наближення камери),
об'єднуються, а треки, що весь час лежать усередині рамки іншого об'єкта
того самого класу, відкидаються. Так само відкидаються «крайові» треки:
низька середня впевненість (< 0.5) і ≥ 80 % кадрів рамка впирається в
край кадру (напр. піднята рука людини, що потрапила в кадр лише частково).


**Водій** — трек класу ``person``, обличчя якого в середньому найбільш
схоже на еталонне фото, якщо середня схожість вища за поріг SFace.
"""
from __future__ import annotations


from collections import Counter, defaultdict
from dataclasses import asdict, dataclass, field


import numpy as np
```

```
from . import config as C
from .camera_motion import CameraMotionEstimator
from .plates import vote


ROLE_DRIVER = "ВОДІЙ"
ROLE_PERSON = "інша людина (пасажир)"
ROLE_CAR_PLATE = "автомобіль з номером"
ROLE_CAR = "інший автомобіль"
ROLE_EXTRA = "додатковий об'єкт"



@dataclass
class TrackSummary:
    track_id: int
    cls: str
    n_frames: int
    first_frame: int
    last_frame: int
    mean_conf: float
    move_score: float          # з компенсацією руху камери
    move_score_naive: float    # як у демонстрації: без компенсації (перша → остання поява)
    move_score_naive_max: float  # без компенсації, максимальне зміщення від першої появи
    moving: bool
    moving_naive: bool
    face_samples: int = 0
    mean_similarity: float | None = None
    role: str = ""
    similarities: list = field(default_factory=list, repr=False)



def _box_to_ref(box, T) -> np.ndarray:
    """Переводить рамку в систему координат першого кадру (обхоплююча рамка 4 кутів)."""
    x1, y1, x2, y2 = box
    p = CameraMotionEstimator.apply(T, [[x1, y1], [x2, y1], [x2, y2], [x1, y2]])
    return np.array([p[:, 0].min(), p[:, 1].min(), p[:, 0].max(), p[:, 1].max()])



def movement_score(samples, frame_wh) -> float:
    """Нормований розмах положення рамки після компенсації руху камери.

    ``samples`` — список пар (рамка в кадрі, матриця «кадр → перший кадр»).
    """
    W, H = frame_wh
    m = C.BORDER_MARGIN
    refs = [_box_to_ref(b, T) for b, T in samples]
    areas = np.array([max(r[2] - r[0], 1) * max(r[3] - r[1], 1) for r in refs])
    # Спостереження, де об'єкт видно лише частково (обрізаний краєм кадру
    # чи перекритий) мають помітно меншу площу — їх рамка ненадійна.
    full = areas >= 0.7 * np.percentile(areas, 90)
    edges: list[list[float]] = [[], [], [], []]
    sizes = []
    for (box, _), ref, ok in zip(samples, refs, full):
        if not ok:
            continue
        sizes.append(np.sqrt(max(ref[2] - ref[0], 1) * max(ref[3] - ref[1], 1)))
        x1, y1, x2, y2 = box
        truncated = [x1 <= m, y1 <= m, x2 >= W - m, y2 >= H - m]
        for i in range(4):
            if not truncated[i]:
                edges[i].append(ref[i])
    size = float(np.median(sizes))
    ranges = [np.percentile(e, 90) - np.percentile(e, 10) for e in edges if len(e) >= 5]
    if not ranges:  # об'єкт увесь час обрізаний краєм кадру — беремо центри
        c = np.array([[(b[0] + b[2]) / 2, (b[1] + b[3]) / 2] for b in (_box_to_ref(b, T)
for b, T in samples)])
        ranges = [float(np.ptp(c[:, 0])), float(np.ptp(c[:, 1]))]
    return float(np.median(ranges) / size)
```

```
def naive_movement_score(samples) -> float:
    """Оцінка з демонстрації: зміщення центру (перша → остання поява) / розмір."""
    (b0, _), (b1, _) = samples[0], samples[-1]
    c0 = np.array([(b0[0] + b0[2]) / 2, (b0[1] + b0[3]) / 2])
    c1 = np.array([(b1[0] + b1[2]) / 2, (b1[1] + b1[3]) / 2])
    size = np.sqrt((b0[2] - b0[0]) * (b0[3] - b0[1]))
    return float(np.linalg.norm(c1 - c0) / size)



def naive_max_movement_score(samples) -> float:
    """Без компенсації: максимальне зміщення центру від першої появи / розмір."""
    b0 = samples[0][0]
    c0 = np.array([(b0[0] + b0[2]) / 2, (b0[1] + b0[3]) / 2])
    size = np.sqrt((b0[2] - b0[0]) * (b0[3] - b0[1]))
    c = np.array([[(b[0] + b[2]) / 2, (b[1] + b[3]) / 2] for b, _ in samples])
    return float(np.linalg.norm(c - c0, axis=1).max() / size)



def _iou(a, b) -> float:
    ix = max(0.0, min(a[2], b[2]) - max(a[0], b[0]))
    iy = max(0.0, min(a[3], b[3]) - max(a[1], b[1]))
    inter = ix * iy
    return inter / ((a[2] - a[0]) * (a[3] - a[1]) + (b[2] - b[0]) * (b[3] - b[1]) - inter +
1e-9)



def merge_fragments(records: list[dict], max_overlap: float = 0.2, min_iou: float = 0.6) ->
dict[int, int]:
    """Об'єднує фрагменти треку одного нерухомого об'єкта.


    Два треки одного класу вважаються одним об'єктом, якщо вони майже не
    співіснують у часі (перекриття ≤ ``max_overlap`` коротшого треку), а їх
    медіанні рамки в системі координат першого кадру збігаються (IoU ≥
    ``min_iou``). Номери треків у ``records`` замінюються на номер
    першого фрагмента. Повертає відображення «старий id → новий id».
    """
    info = defaultdict(lambda: {"frames": set(), "boxes": [], "cls": Counter()})
    for rec in records:
        for d in rec["dets"]:
            t = info[d.track_id]
            t["frames"].add(rec["frame"])
            t["boxes"].append(_box_to_ref(d.box, rec["T"]))
            t["cls"][d.cls] += 1
    med = {k: np.median(v["boxes"], axis=0) for k, v in info.items()}
    cls = {k: v["cls"].most_common(1)[0][0] for k, v in info.items()}
    mapping: dict[int, int] = {}
    ids = sorted(info, key=lambda k: min(info[k]["frames"]))
    for i, a in enumerate(ids):
        root_a = mapping.get(a, a)
        for b in ids[i + 1:]:
            if b in mapping or cls[a] != cls[b]:
                continue
            fa, fb = info[root_a]["frames"], info[b]["frames"]
            overlap = len(fa & fb) / min(len(fa), len(fb))
            if overlap <= max_overlap and _iou(med[root_a], med[b]) >= min_iou:
                mapping[b] = root_a
                info[root_a]["frames"] |= fb
    for rec in records:
        for d in rec["dets"]:
            d.track_id = mapping.get(d.track_id, d.track_id)
    return mapping



def part_of_other(per_track: dict, containment: float = 0.9, share: float = 0.8) ->
set[int]:
    """Треки, що в ≥ ``share`` своїх кадрів лежать усередині (≥ ``containment``
    площі) рамки іншого, довшого треку того самого класу — це частина
```

```
    іншого об'єкта (напр. рука людини), а не окремий об'єкт."""
    by_frame = defaultdict(list)
    for tid, items in per_track.items():
        for f, d, _ in items:
            by_frame[f].append(d)
    length = {tid: len(items) for tid, items in per_track.items()}
    parts = set()
    for tid, items in per_track.items():
        inside = 0
        for f, d, _ in items:
            a = d.box
            area = (a[2] - a[0]) * (a[3] - a[1])
            for o in by_frame[f]:
                if o.track_id == tid or o.cls != d.cls or length[o.track_id] <=
length[tid]:
                    continue
                b = o.box
                ix = max(0.0, min(a[2], b[2]) - max(a[0], b[0]))
                iy = max(0.0, min(a[3], b[3]) - max(a[1], b[1]))
                if ix * iy >= containment * area:
                    inside += 1
                    break
        if inside >= share * len(items):
            parts.add(tid)
    return parts



def _contains(box, pt) -> bool:
    return box[0] <= pt[0] <= box[2] and box[1] <= pt[1] <= box[3]



def summarize(records: list[dict], frame_wh: tuple[int, int]) -> dict:
    """Агрегує покадрові записи конвеєра в підсумкові рішення.


    Кожен запис ``records[i]`` містить: ``dets`` (список Detection),
    ``T`` (матриця «кадр → перший кадр»), ``plates`` і ``faces``
    (списки або ``None``, якщо кадр не аналізувався на номер/обличчя).
    """
    n_frames = len(records)
    merged = merge_fragments(records)
    per_track = defaultdict(list)                # id → [(frame, Detection, T)]
    sims = defaultdict(list)                     # id → [схожість обличчя]
    plate_owner = Counter()                      # id авто → скільки разів номер у його
рамці
    plate_reads = []


    for rec in records:
        for d in rec["dets"]:
            per_track[d.track_id].append((rec["frame"], d, rec["T"]))


        # Обличчя прив'язуємо до найменшої рамки person, що містить центр обличчя
        for face in rec["faces"] or []:
            owners = [d for d in rec["dets"] if d.cls == "person" and _contains(d.box,
face.center)]
            if owners:
                owner = min(owners, key=lambda d: (d.box[2] - d.box[0]) * (d.box[3] -
d.box[1]))
                sims[owner.track_id].append(face.similarity)


        # Номер прив'язуємо до автомобіля, в рамці якого він знаходиться
        for pr in rec["plates"] or []:
            plate_reads.append(pr)
            c = ((pr.box[0] + pr.box[2]) / 2, (pr.box[1] + pr.box[3]) / 2)
            cars = [d for d in rec["dets"] if d.cls in ("car", "truck", "bus") and
_contains(d.box, c)]
            if cars:
                plate_owner[min(cars, key=lambda d: (d.box[2] - d.box[0]) * (d.box[3] -
d.box[1])).track_id] += 1
```

```
    min_len = C.MIN_TRACK_FRACTION * n_frames
    parts = part_of_other(per_track)
    W, H = frame_wh
    m = C.BORDER_MARGIN
    edge_tracks = set()
    for tid, items in per_track.items():
        at_border = np.mean([d.box[0] <= m or d.box[1] <= m or d.box[2] >= W - m or
d.box[3] >= H - m
                             for _, d, _ in items])
        if np.mean([d.conf for _, d, _ in items]) < 0.5 and at_border >= 0.8:
            edge_tracks.add(tid)
    tracks: list[TrackSummary] = []
    for tid, items in sorted(per_track.items()):
        if len(items) < min_len or tid in parts or tid in edge_tracks:
            continue
        cls = Counter(d.cls for _, d, _ in items).most_common(1)[0][0]
        samples = [(d.box, T) for _, d, T in items]
        score = movement_score(samples, frame_wh)
        naive = naive_movement_score(samples)
        naive_max = naive_max_movement_score(samples)
        s = sims.get(tid, [])
        tracks.append(TrackSummary(
            track_id=tid, cls=cls, n_frames=len({f for f, _, _ in items}),
first_frame=items[0][0], last_frame=items[-1][0],
            mean_conf=float(np.mean([d.conf for _, d, _ in items])),
            move_score=score, move_score_naive=naive, move_score_naive_max=naive_max,
            moving=score > C.MOVING_THRESHOLD, moving_naive=naive > C.MOVING_THRESHOLD,
            face_samples=len(s), mean_similarity=float(np.mean(s)) if s else None,
similarities=s))


    # Водій: найбільша середня схожість серед людей з достатньою кількістю облич
    candidates = [t for t in tracks if t.cls == "person" and t.face_samples >=
C.MIN_FACE_SAMPLES]
    driver = max(candidates, key=lambda t: t.mean_similarity, default=None)
    if driver is not None and driver.mean_similarity <= C.SFACE_COSINE_THRESHOLD:
        driver = None


    plate = vote(plate_reads)
    plate_text = plate["by_char"] or plate["by_string"]
    plate_car = plate_owner.most_common(1)[0][0] if plate_owner else None


    for t in tracks:
        if t.cls == "person":
            t.role = ROLE_DRIVER if driver is not None and t.track_id == driver.track_id
else ROLE_PERSON
        elif t.cls in ("car", "truck", "bus"):
            t.role = f"{ROLE_CAR_PLATE} {plate_text}" if t.track_id == plate_car else
ROLE_CAR
        else:
            t.role = ROLE_EXTRA


    return {
        "n_frames": n_frames,
        "n_tracks_total": len(per_track),
        "merged_tracks": {int(k): int(v) for k, v in merged.items()},
        "part_tracks": sorted(int(t) for t in parts),
        "edge_tracks": sorted(int(t) for t in edge_tracks),
        "tracks": tracks,
        "moving_ids": [t.track_id for t in tracks if t.moving],
        "moving_ids_naive": [t.track_id for t in tracks if t.moving_naive],
        "driver_id": driver.track_id if driver else None,
        "driver_similarity": driver.mean_similarity if driver else None,
        "plate": plate,
        "plate_text": plate_text,
        "plate_car_id": plate_car,
    }



def to_jsonable(summary: dict) -> dict:
    """Готує підсумок до запису в JSON (dataclass → dict)."""
    out = dict(summary)
```

```
    out["tracks"] = [asdict(t) for t in summary["tracks"]]
    return out
```

Лістинг А.7 – Модуль pipeline.py

```
"""Повний конвеєр обробки відео.

Схема (для кожного кадру):

    кадр ─┬─► YOLO11m + ByteTrack ─► об'єкти з номерами треків
          ├─► оптичний потік + RANSAC ─► рух камери (кадр → перший кадр)
          └─► (кожен PROCESS_EVERY-й кадр)
                ├─► YOLOv11-LP ─► рамка номера ─► збільшення, CLAHE ─► EasyOCR ─► шаблон UA
                └─► YuNet ─► вирівнювання ─► SFace ─► схожість з еталоном водія

Після проходу по відео — агрегування по треках (:mod:`analysis`),
анотоване відео (:mod:`annotate`) і, якщо є еталонна розмітка, оцінка
якості (:mod:`evaluation`).
"""
from __future__ import annotations

import csv
import json
import pickle
import time
from pathlib import Path

import cv2
import numpy as np

from . import config as C
from .analysis import summarize, to_jsonable
from .annotate import write_annotated
from .camera_motion import CameraMotionEstimator
from .detection import ObjectTracker
from .evaluation import evaluate
from .faces import FaceIdentifier
from .plates import PlateReader
from .report_figures import build_all

SNAPSHOT_FRAMES = (60, 300, 380, 520, 700, 900)



def process_video(video: Path, driver_photo: Path, verbose: bool = True) ->
tuple[list[dict], dict]:
    """Перший прохід: покадрова обробка. Повертає (записи по кадрах, метадані відео)."""
    tracker = ObjectTracker()
    plates = PlateReader()
    faces = FaceIdentifier()
    faces.set_reference(driver_photo)
    camera = CameraMotionEstimator()


    cap = cv2.VideoCapture(str(video))
    meta = {"fps": cap.get(cv2.CAP_PROP_FPS), "frames":
int(cap.get(cv2.CAP_PROP_FRAME_COUNT)),
            "width": int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)), "height":
int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))}
    records, f, t0 = [], 0, time.time()
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        dets = tracker.update(frame)
        T = camera.update(frame, [d.box for d in dets])
        rec = {"frame": f, "t": f / meta["fps"], "dets": dets, "T": T.copy(), "plates":
None, "faces": None}
```

```
        if f % C.PROCESS_EVERY == 0:
            rec["plates"] = plates.read(frame)
            rec["faces"] = faces.process(frame)
        records.append(rec)
        f += 1
        if verbose and f % 100 == 0:
            print(f"  оброблено {f}/{meta['frames']} кадрів ({f / (time.time() - t0):.1f}
кадр/с)")
    cap.release()
    meta["processing_s"] = time.time() - t0
    meta["camera_inliers_mean"] = float(np.mean(camera.inliers[1:])) if len(camera.inliers)
> 1 else 0.0
    meta["reference_face"] = faces.reference_face
    return records, meta



def save_frame_log(records: list[dict], path: Path) -> None:
    """Зберігає всі покадрові результати у CSV (об'єкти, номери, обличчя)."""
    with path.open("w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["frame", "t", "kind", "track_id", "cls", "conf", "x1", "y1", "x2",
"y2", "text_raw",
                    "plate", "similarity"])
        for r in records:
            for d in r["dets"]:
                w.writerow([r["frame"], f"{r['t']:.3f}", "object", d.track_id, d.cls,
f"{d.conf:.3f}",
                            *[f"{v:.1f}" for v in d.box], "", "", ""])
            for p in r["plates"] or []:
                w.writerow([r["frame"], f"{r['t']:.3f}", "plate", "", "plate",
f"{p.det_conf:.3f}",
                            *[f"{v:.1f}" for v in p.box], p.raw_text, p.plate or "", ""])
            for fc in r["faces"] or []:
                w.writerow([r["frame"], f"{r['t']:.3f}", "face", "", "face",
f"{fc.score:.3f}",
                            *[f"{v:.1f}" for v in fc.box], "", "", f"{fc.similarity:.4f}"])



def print_summary(video: Path, summary: dict) -> None:
    """Текстовий підсумок у стилі демонстрації."""
    print(f"\nПІДСУМОК ОБРОБКИ ВІДЕО {video.name}\n" + "-" * 70)
    print(f"Треків всього: {summary['n_tracks_total']}, об'єднано фрагментів:
{summary['merged_tracks']}, "
          f"частини інших об'єктів: {summary['part_tracks']}, крайові:
{summary['edge_tracks']}, "
          f"залишилось: {len(summary['tracks'])}")
    print(f"{'трек':>4} {'клас':8} {'кадрів':>6} {'впевн.':>6} {'рух*':>6} {'наїв.1':>7}
{'наїв.2':>7} "
          f"{'рухається':>9}  {'схожість з водієм':22} роль")
    for t in summary["tracks"]:
        sim = f"{t.mean_similarity:.3f} (у {t.face_samples} кадрах)" if t.mean_similarity
is not None else "-"
        print(f"{t.track_id:>4} {t.cls:8} {t.n_frames:>6} {t.mean_conf:6.2f}
{t.move_score:6.2f} "
              f"{t.move_score_naive:7.2f} {t.move_score_naive_max:7.2f} {'так' if t.moving
else 'ні':>9}  "
              f"{sim:22} {t.role}")
    print("* рух — з компенсацією руху камери; наїв.1 — як у демонстрації (перша→остання
поява);")
    print("  наїв.2 — максимальне зміщення без компенсації; поріг", C.MOVING_THRESHOLD)
    p = summary["plate"]
    print(f"\nНомерний знак: голоси {p['counts']}")
    print(f"  за рядком: {p['by_string']}, посимвольно: {p['by_char']} ->
{summary['plate_text']}")
    if summary["driver_id"] is not None:
        print(f"Водій: трек #{summary['driver_id']}, середня схожість
{summary['driver_similarity']:.3f} "
              f"> {C.SFACE_COSINE_THRESHOLD}")
    else:
        print("Водія не ідентифіковано")
```

```
RECORDS_CACHE = C.DATA_DIR / "records.pkl"



def run_pipeline(video: Path = C.SCENE_VIDEO, driver_photo: Path = C.DRIVER_REFERENCE,
                 gt_path: Path | None = C.SCENE_GT, verbose: bool = True, reuse: bool =
False) -> dict:
    """Запускає всі кроки і зберігає результати в ``data/`` та ``report/figures/``.


    ``reuse=True`` бере покадрові результати нейромереж з кешу
    (``data/records.pkl``) і перераховує лише аналіз — зручно, коли
    змінюються пороги чи правила агрегування.
    """
    C.DATA_DIR.mkdir(exist_ok=True)
    C.FIGURES_DIR.mkdir(parents=True, exist_ok=True)
    if reuse and RECORDS_CACHE.exists():
        records, meta = pickle.loads(RECORDS_CACHE.read_bytes())
        if verbose:
            print(f"Покадрові результати взято з кешу {RECORDS_CACHE.name}")
    else:
        if verbose:
            print(f"Обробка {video} ...")
        records, meta = process_video(video, driver_photo, verbose)
        RECORDS_CACHE.write_bytes(pickle.dumps((records, meta)))
    if verbose:
        print(f"Оброблено {len(records)} кадрів за {meta['processing_s']:.0f} с "
              f"({len(records) / meta['processing_s']:.1f} кадр/с)")


    summary = summarize(records, (meta["width"], meta["height"]))
    write_annotated(video, C.ANNOTATED_VIDEO, records, summary, SNAPSHOT_FRAMES,
C.FIGURES_DIR)
    save_frame_log(records, C.FRAMES_CSV)
    if meta["reference_face"] is not None:
        cv2.imwrite(str(C.FIGURES_DIR / "reference_face_aligned.png"),
meta["reference_face"])


    result = {"video": str(video.name), "meta": {k: v for k, v in meta.items() if k !=
"reference_face"},
              "summary": to_jsonable(summary)}
    gt = None
    if gt_path is not None and gt_path.exists():
        gt = json.loads(gt_path.read_text())
        if gt["video"] == video.name:
            result["evaluation"] = evaluate(records, summary, gt)
        else:
            gt = None
    C.RESULTS_JSON.write_text(json.dumps(result, ensure_ascii=False, indent=1,
default=float))
    figures = build_all(video, records, summary, result, gt, driver_photo)
    if verbose:
        print("Ілюстрації:", ", ".join(f.name for f in figures))


    if verbose:
        print_summary(video, summary)
        if "evaluation" in result:
            ev = result["evaluation"]
            print("\nОЦІНКА ЗА ЕТАЛОННОЮ РОЗМІТКОЮ")
            for name, d in ev["detection"].items():
                print(f"  {name:11} recall@IoU0.5={d['recall_iou50']}  mean
IoU={d['mean_iou']}  треки={d['track_ids']}")
            print(f"  номер: {ev['plate']}")
            print(f"  водій: {ev['driver']['predicted_is']} (правильно: {ev['driver']
['correct']})")
            print(f"  рухомий: {ev['moving']['predicted']} (правильно: {ev['moving']
['correct']}); "
                  f"наїв.1: {ev['moving']['naive_predicted']}; наїв.2: {ev['moving']
['naive_max_predicted']}")
            print(f"  похибка оцінки руху камери: середня {ev['camera']['mean_error_px']}
px, "
                  f"макс. {ev['camera']['max_error_px']} px")
```

```
        print(f"\nАнотоване відео: {C.ANNOTATED_VIDEO}")
    return {"records": records, "summary": summary, "meta": meta, "result": result}
```

ДОДАТОК Б

Тези доповіді про виконану роботу

1) **Завдання.** Обробити відео 30–60 с: виділити й розпізнати об'єкти, прочитати номер автомобіля, ідентифікувати водія, визначити рухомий об'єкт;

2) **Відео.** Синтетична 40-секундна сцена: ЗАЗ Sens з номером КА 7429 МВ, другий автомобіль, водій, пасажир, що йде (рухомий об'єкт), собака; камера «з руки» з наближенням; є еталонна розмітка;

3) **Конвеєр.** YOLO11m + ByteTrack → об'єкти й треки; оптичний потік + RANSAC → рух камери; YOLOv11-LP + EasyOCR + шаблон + голосування → номер; YuNet + SFace → водій;

4) **Результати.** 5 об'єктів знайдено правильно; номер KA7429MB прочитано точно в 100 % кадрів; водій — схожість 0,71 проти 0,12; рухомий — пасажир (оцінка 1,42 при порозі 0,25);

5) **Головний висновок.** Без компенсації руху камери нерухомі об'єкти визнаються рухомими; рішення по треку та постобробка OCR роблять систему стійкою до помилок окремих кадрів;

6) **Демонстрація.** Анотоване відео data/synthetic_scene_annotated.mp4 з рамками, позначками DRIVER і MOVING, номером і траєкторією рухомого об'єкта.